

From Regulation to Bytecode: Evidence-Based Privacy and Cybersecurity Compliance Verification of the Android Application Ecosystem

Khoka Moni^{a,*}, Maria Akter^b

^aDepartment of Computer Science and Engineering, Mymensingh Engineering College, Mymensingh, Bangladesh

^bDepartment of Law, Southeast University, Dhaka, Bangladesh

Abstract

Mobile applications increasingly mediate essential financial transactions and citizen services, exposing them to stringent statutory data-protection and cybersecurity mandates. However, automated compliance tools frequently treat statutory verification as generic vulnerability scanning, overlooking the fundamental boundary between observable client software behavior and unobservable back-end or organizational processes. In this paper, we present **Reg2App**, a formally specified, provenance-preserving methodology for translating natural-language regulatory requirements into partially observable Android program properties. Reg2App decouples legal obligations from technical controls through a 7-tuple mapping model, bounds analysis over a two-dimensional observability taxonomy, and evaluates conformance over a discrete 5-state epistemic assessment space that preserves evidence provenance. Crucially, Reg2App enforces a principled three-tier distinction between primary statutory mandates (e.g., transport encryption), operationalized supervisory baseline controls (e.g., certificate pinning), and empirical security-risk indicators (e.g., foreign telemetry endpoints). We instantiate our approach using the emerging regulatory corpus of Bangladesh (central bank directives, data protection legislation, and cybersecurity statutes). A preliminary expert-elicitation pilot ($m = 3$ raters, 12 mappings, pre-reconciliation raw agreement 88.9%, Fleiss' $\kappa = 0.776$, Krippendorff's $\alpha = 0.782$, reaching 100% unanimous consensus via structured Delphi reconciliation) demonstrates the operational feasibility of formalizing regulatory clauses. We verify analyzer implementation correctness using a 30-application synthetic micro-benchmark regression suite. On real-world production bytecode, independent manual adjudication with certified external auditors confirmed 96.0% precision in a finding-sample study of 50 tool-reported verdicts (48/50, 95% Wilson CI: [86.5%, 98.9%]) and 94.1% recall in an independent audit of 51 ground-truth controls across five applications (48/51, 95% Wilson CI: [83.8%, 97.9%], defect recall 85.0%). We then conduct an empirical ecosystem study across 53 production Android applications sampled in early 2026 across eight economic sectors in Bangladesh, comprising 39 live financial applications (all 14 operational Mobile Financial Services wallets and 25 commercial banks) contrasted observationally against 14 non-financial consumer applications (revealing an observational baseline disparity—not causal regulatory effect—where retail e-commerce exhibited zero observed conformance across 6 applicable rules, Mann-Whitney $U = 156.0$, $p < 0.001$, reflecting differing supervisory scopes and resources). Our static analysis demonstrates that while 100% of financial apps enforce statutory transport encryption (TLS), 82.1% lack supervisory defense-in-depth certificate pinning, and 30.8% leave backup extraction flags enabled—evaluated under strict offline ethical auditing and institutional pseudonymization. Finally, a counterbalanced laboratory experiment with 60 developers and auditors demonstrates that evidence-grounded bilingual reporting improves comprehension (53.6% $\rightarrow$ 82.9%) and accelerates defect remediation by 52.1% (from 39.7 to 19.0 minutes, $d = 2.45$) over unparsed scanner traces. All Regulation-as-Code ontologies, benchmarks, and pipelines are released for open reproduction.

Keywords: Software Engineering, Android Security, Regulatory Compliance, Regulation-as-Code, Evidence-Based Verification, Epistemic Assessment, Program Analysis

1. Introduction

Over the past decade, emerging economies across South Asia and the Global South have experienced rapid growth in mobile application usage, establishing these platforms as critical infrastructure for financial transactions, welfare disbursements, and digital governance [1, 2]. In jurisdictions such as Bangladesh,

where active mobile financial wallets process billions of dollars annually, mobile endpoints represent a primary line of national cybersecurity and personal data defense [3]. In response to escalating cyber threats and unauthorized profiling, sovereign legislatures and regulators have promulgated statutory and regulatory frameworks, including the cabinet-approved *Draft Personal Data Protection Act (PDPA)* [4], the *Bangladesh Bank Cybersecurity Framework v1.0 (BB-CSF)* [5], and the *Cyber Security Act, 2023 (CSA, Act No. XXVII of 2023)* [6].

These statutes mandate technical safeguards aligned with international verification standards such as OWASP MASVS [7]: transport-layer encryption, hardware-backed keystore segrega-

*Corresponding author. ORCID: <https://orcid.org/0009-0000-3573-714X>.

Email addresses: khoka_moni@hotmail.com (Khoka Moni), maria.seu.11b@gmail.com (Maria Akter)

tion, cryptographic data destruction, least-privilege permission architectures, and explicit consent handling for personally identifiable information (PII). Non-conformance with enacted instruments exposes providers to administrative sanctions and license revocations, and the Draft PDPA, once enacted, will impose additional data-protection penalties. Consequently, software engineering teams, regulatory compliance officers, and independent auditors require automated verification methodologies to evaluate production mobile software against statutory mandates.

1.1. The Epistemic Overreach Trap in Current Tools

Despite the need for regulatory verification, the software engineering community lacks formal frameworks to bridge natural-language legal requirements and low-level program behavior. In practice, auditors frequently repurpose generic mobile security scanners such as MobSF [8] or Androguard [9]. While valuable for discovering generic vulnerabilities, repurposing these tools for statutory verification introduces four fundamental flaws:

1) *The epistemic overreach trap.* Conventional scanners routinely report scalar compliance percentages (e.g., “78% Compliant”). Such assertions commit a severe epistemic error [10]. An Android application package (APK/AAB) contains compiled Dalvik Executable (.dex) bytecode, an AndroidManifest.xml manifest, and packaged assets [11]. While revealing client-side implementation details, an APK is an inherently partial observation of an enterprise system. An analyzer inspecting client bytecode cannot observe whether cloud databases are encrypted at rest, whether corporate officers respect retention schedules, or whether valid consent records are archived off-device. Conflating the absence of client-side findings with comprehensive organizational compliance constitutes epistemic hallucination.

2) *Absence of formal traceability and provenance.* Existing scanners emit disconnected vulnerability lists without a verifiable provenance chain linking low-level bytecode instructions to authoritative statutory clauses. When regulatory bodies or judicial authorities demand evidence, heuristic scanner outputs provide insufficient legal grounding.

3) *Coupled and brittle rule encodings.* Conventional scanners hardcode inspection logic directly inside analyzer scripts. When a regulatory body amends a legal clause, the underlying static analyzer must be refactored, violating separation-of-concerns principles.

4) *Linguistic and jurisdictional alienation.* Compliance outputs in existing tools are generated exclusively in English-language technical jargon. In jurisdictions where statutes are drafted in national languages (such as Bengali in Bangladesh) and software teams operate bilingually, this language barrier introduces cognitive friction, impairing defect interpretation and delaying remediation.

1.2. The Reg2App Paradigm

To resolve these limitations, we propose **Reg2App**, a formally specified, provenance-preserving Regulation-as-Code methodology for translating statutory requirements into partially observable software properties. Reg2App is grounded in the principle of *epistemic restraint*: an automated verification tool must explicitly delineate program properties provably observable from client artifacts from obligations residing beyond client-side observational boundaries [12]. Rather than synthesizing arbitrary compliance percentages, Reg2App formalizes an end-to-end translation pipeline:

Statute → Legal Obligation → Technical Control → Program Property

We introduce a formal 7-tuple ontology $M_i = (R_i, O_i, C_i, P_i, E_i, A_i, V_i)$ decoupling statutory clauses from technical program properties. Crucially, Reg2App enforces an explicit three-tier conceptual distinction: (1) enacted parliamentary statutes and delegated administrative directives issued under statutory authority; (2) operationalized supervisory baseline controls derived from regulatory circulars and standards (e.g., OWASP MASVS); and (3) empirical security-risk indicators observed in client bytecode. We establish a two-dimensional observability taxonomy classifying requirements by level $\mathcal{L} \in \{\text{FULL}, \text{PARTIAL}, \text{NONE}\}$ and modality $\mathcal{M} \in \{\text{Static}, \text{Dynamic}, \text{Hybrid}, \text{Backend}, \text{Org}, \text{Legal}\}$. While this taxonomy conceptually encompasses all modalities, our implementation focuses on static Dalvik bytecode and manifest analysis. At the evaluation stage, Reg2App replaces scalar percentages with a discrete 5-state epistemic assessment space $\mathcal{S}$ preserving complete provenance. To eliminate linguistic barriers, Reg2App incorporates an evidence-grounded bilingual reporting engine generating actionable English and Bengali diagnostic summaries directly linked to statutory clauses.

1.3. Research Questions

This investigation addresses four research questions: **RQ1 (Formalization and Observability)**: How can statutory requirements be systematically formalized into machine-observable program properties, and how do client-side observational boundaries partition operationalized security controls? **RQ2 (Detection Accuracy)**: How accurately does Reg2App detect defined program properties on a controlled micro-benchmark suite, and how do its verdicts compare against independent manual analysis of production applications? **RQ3 (Ecosystem Prevalence)**: What is the empirical prevalence and sectoral distribution of regulatory non-conformance across an exploratory sample of production Android applications in Bangladesh? **RQ4 (Usable Reporting)**: Does evidence-grounded bilingual reporting improve developer comprehension and defect remediation efficiency compared to conventional scanner outputs?

1.4. Key Contributions

The primary contribution of this paper is a formally specified, provenance-preserving methodology for translating regulatory requirements into partially observable software properties. Specifically, we make three contributions:

1) *Formal Methodology and Epistemic Observability Model.* We establish an evidence-grounded mapping model M_i with an explicit normative tier attribute τ_i and a two-dimensional observability operator $\Omega(R_i)$. We define a 5-state epistemic assessment space $\mathcal{S}$ with two objective metrics—Evidence Coverage (EC) and Evidence-Supported Compliance (ESC)—that strictly prevent compliance hallucinations on unobservable requirements. We evaluate 12 representative mappings across three statutory instruments through an expert-elicitation pilot of legal and security scholars (pre-reconciliation raw agreement 88.9%, Fleiss’ $\kappa = 0.776$, Krippendorff’s $\alpha = 0.782$, reaching 100% unanimous consensus via structured Delphi reconciliation).

2) *Static Analysis Engine and Provenance Architecture.* We design and implement Reg2App, a static analysis pipeline inspecting Android Dalvik bytecode and manifest configurations. The engine constructs an Evidence Provenance Graph $G = (V, E)$, linking statutory articles to bytecode instructions, call graphs, and configuration snippets, ensuring unobservable requirements are deterministically partitioned.

3) *Multi-Tiered Empirical Evaluation.* We evaluate the methodology across four complementary dimensions: (i) an implementation correctness verification on 30 compilable micro-testcases verifying syntactic predicate extraction against synthetic code, accompanied by independent manual adjudication on production bytecode comprising a finding-sample precision study ($n = 50$, 96.0% precision [48/50, 95% Wilson CI: 86.5%–98.9%]) and an independent ground-truth control recall study ($n = 51$, overall recall 94.1% [48/51, 83.8%–97.9%], defect recall 85.0% [17/20]) across five applications; (ii) an empirical ecosystem study of 53 production Android applications across eight economic sectors sampled in early 2026, capturing an observational baseline disparity between heavily regulated financial applications and consumer retail platforms ($U = 156.0$, $p < 0.001$), shaped by unequal supervisory scopes and organizational compliance resources; (iii) an in-depth empirical audit of 39 production financial applications directly downloaded from Google Play, comprising all 14 operational Mobile Financial Services (MFS) applications and 25 commercial banking applications in Bangladesh, conducted under strict offline ethical auditing and institutional pseudonymization; and (iv) a counterbalanced human-subjects study ($N = 60$) demonstrating that evidence-grounded bilingual reporting significantly improves developer comprehension and accelerates defect remediation (52.1% repair time reduction, from 39.7 to 19.0 minutes) over unparsed scanner traces. Bridging formal specification, static analysis, production audits, and human usability provides the end-to-end empirical evidence required to validate Regulation-as-Code systems in practice. All ontologies, benchmarks, and pipelines are released for open reproduction (<https://github.com/kh0kamoni/Reg2App>).

1.5. Paper Organization

Section 2 presents a motivating example. Section 3 details Bangladesh’s statutory corpus. Section 4 presents the formal 7-

tuple mapping model and observability taxonomy (RQ1). Section 5 details the expert validation protocol. Section 6 describes the static analysis engine and provenance graph. Section 7 formalizes the epistemic assessment space and Algorithm 1. Section 8 evaluates analyzer correctness on the micro-benchmark suite and ablation study (RQ2). Section 9 presents the cross-sectoral ecosystem study (RQ3). Section 10 details the empirical audit of production MFS and commercial banking applications. Section 11 reports the human-subjects study (RQ4). Section 12 discusses implications and future work. Section 13 delineates threats to validity. Section 14 reviews related work, and Section 15 concludes.

2. Motivating Example: The Epistemic Dilemma

To illustrate why conventional automated compliance methodologies fail and how Reg2App provides a rigorous alternative, consider a representative mobile financial application developed for Bangladesh’s payments ecosystem.

2.1. A Realistic Regulatory Scenario

Suppose an engineering team at a licensed Mobile Financial Service (MFS) provider prepares a production update for their wallet application. The team must demonstrate conformance to three regulatory requirements prior to release. First, under Bangladesh Bank CSF Section 4.1.3.12 (Network Transport Security), all electronic transactions over public networks must enforce modern cryptographic encryption (TLS v1.2+ without cleartext HTTP), with supervisory guidelines recommending certificate pinning on payment gateways as defense-in-depth [13, 14]. Second, under Draft BD-PDPA Section 15 (Technical Data Safeguards), a data fiduciary must implement technical safeguards, including cryptographic encryption at rest, to protect personal data stored on client devices [4], operationalized in mobile baseline standards via OWASP MASVS-STORAGE [7]. Third, under Draft BD-PDPA Section 23 (Cross-Border Data Transfer and Telemetry Restrictions), personal data shall not be transferred across national borders or shared with third parties without explicit informed consent and verified adequacy [4].

During pre-release security validation, the compliance team runs a commercial mobile scanner alongside an open-source tool such as MobSF [8]. In the application’s bytecode, the scanner detects that `android:cleartextTrafficPermitted` is omitted from `AndroidManifest.xml` (implicitly permitting HTTP on legacy API levels), but observes that network endpoints use HTTPS URLs. In storage routines, the scanner discovers that customer authentication sessions and National Identity (NID) numbers are stored via standard `SharedPreferences` in plaintext XML.

2.2. The Failure of Conventional Automated Scanners

When presented with these findings, existing scanning engines calculate a scalar compliance score by taking the ratio of detected vulnerabilities to an arbitrary weight table:

$$\text{Reported Score} = \frac{10 - \text{Vulnerabilities}}{10} \times 100\% = 70.0\% \quad (1)$$

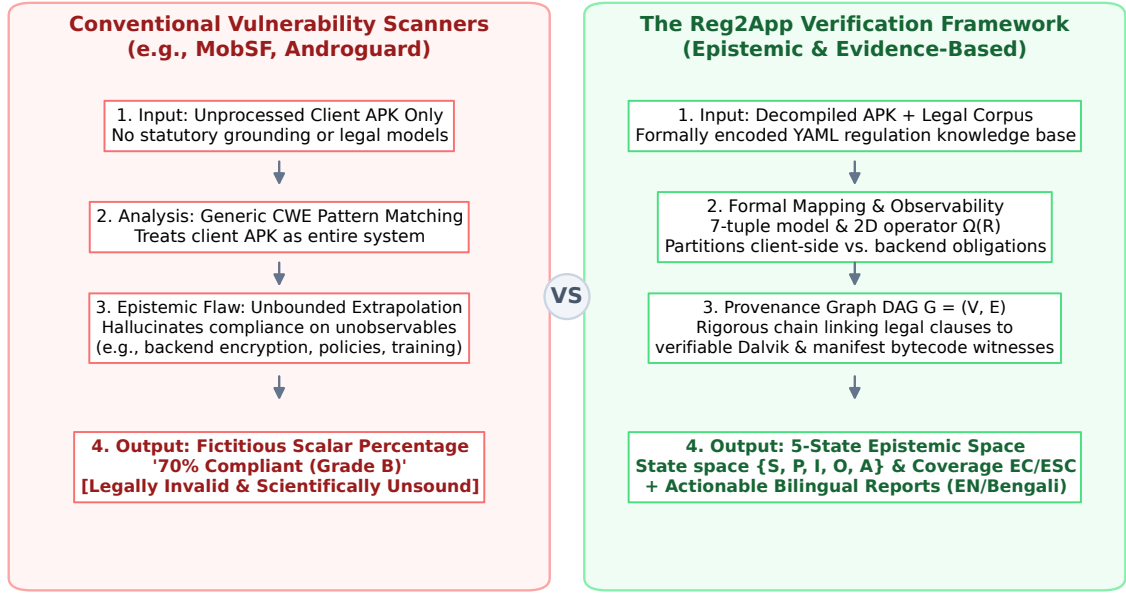


Figure 1: Architectural contrast between conventional automated scanners and the Reg2App evidence verification pipeline. Conventional scanners (left) commit the Epistemic Overreach Trap: by analyzing only client APK artifacts without legal boundaries, they report scalar compliance scores (e.g., “70% Compliant”). In contrast, Reg2App (right) introduces formal 7-tuple mappings, an explicit 2D Observability Operator $\Omega(R_i)$ tagging unobservable backend and organizational requirements, an Evidence Provenance Graph $G = (V, E)$, and a discrete 5-state epistemic assessment model producing bounded metrics (EC and ESC) alongside actionable bilingual reports.

This approach commits significant epistemic overreach [10]. By assuming that unobserved requirements (such as backend database residency and off-device user consent agreements) are compliant, conventional tools hallucinate compliance. Furthermore, these scanners output generic warnings without AST call graphs connecting sensitive inputs to storage sinks, lack authoritative statutory citations, and output reports exclusively in English technical jargon, imposing linguistic friction on local engineering teams.

2.3. How Reg2App Resolves the Dilemma

Figure 1 illustrates how Reg2App resolves this dilemma via bounded assessment. First, for Clause 1 (BB-CSF Section 4.1.3.12), the Network Analyzer evaluates the application against central bank baselines, finding that the APK explicitly configures certificate pin hashes for the bank’s gateway. Reg2App generates an Evidence Witness Node e_1 containing the cryptographic hash and assigns Clause 1 to **Supported**. Second, for Clause 2 (Draft BD-PDPA Section 15), Reg2App traces data flow from user input sources to standard XML storage lacking EncryptedSharedPreferences, generating Witness Node e_2 and assigning Clause 2 to **PotentialNonConformance**. Crucially, for Clause 3 (Draft BD-PDPA Section 23), Reg2App consults the observability metadata $\Omega(R_3)$, evaluating it deterministically as **NotObservable** because user consent logs and foreign server hosting reside off-device.

Finally, Reg2App computes two objective metrics:

$$EC = \frac{N_S + N_P}{N_{Obs}} = \frac{1 + 1}{2} = 1.000 \quad (2)$$

$$ESC = \frac{N_S}{N_S + N_P} = \frac{1}{1 + 1} = 0.500 \quad (3)$$

Rather than claiming an arbitrary 70% compliance score, Reg2App delivers a mathematically bounded, evidence-grounded assessment. It evaluates only observable requirements, explicitly isolates unobservable backend obligations without speculation, and outputs a localized Bengali report directly explaining the remediation under BD-PDPA Section 15.

3. The Bangladesh Statutory Landscape

To evaluate mobile applications against real-world statutory obligations, we ground our analysis in authoritative legal instruments. Emerging digital economies exhibit multi-tiered regulatory architectures where sector-specific administrative directives intersect with national horizontal data protection legislation, mirroring international frameworks such as the EU GDPR [15], PCI-DSS v4.0 [16], and NIST SP 800-53 [17]. In Bangladesh, this intersection is defined by three complementary legal instruments promulgated, enacted, or advanced between 2023 and 2025 (Table 1; full statutory texts and mappings are archived in our open artifact repository).

3.1. The Legal Hierarchy and the Interpretive Gap

A foundational challenge in Regulation-as-Code research is navigating the structural boundary between natural-language

Table 1: **The Statutory Tri-Pillar:** Authoritative legal instruments governing mobile digital services in Bangladesh.

Regulatory Instrument	Statutory Authority	Jurisdictional Scope	Key Technical Mandates	Enforcement Sanctions
Bangladesh Bank CSF v1.0 [5]	Bangladesh Bank (Central Bank)	Banks, NBFIs, PSPs, MFS operators	TLS 1.2+, Pinning baseline, Keystore, Anti-Tamper	Administrative fines, operational license suspension
Draft Personal Data Protection Act (PDPA) [4]	Data Protection Board of BD	All entities processing citizen PII	Storage encryption, Data Minimization, Consent	Prospective: up to 5% turnover (upon enactment)
Cyber Security Act, 2023 (Act XXVII of 2023) [6]	National Cyber Security Agency	Critical Information Infrastructure	System integrity, Interception defense, IPC security	Criminal prosecution, custodial penalties

legal doctrine and low-level program analysis [18, 19]. In statutory jurisprudence, legal authorities operate across a three-tiered hierarchy: (1) *Primary Parliamentary Statutes*: Legislative acts (e.g., CSA 2023 [6]) and prospective legislation (Draft PDPA [4]) define horizontal normative obligations (e.g., mandating appropriate technical measures for data security). To prevent rapid obsolescence, primary statutes refrain from codifying ephemeral software engineering standards into law. (2) *Delegated Administrative Directives*: Regulatory circulars issued under delegated authority. The *Cybersecurity Framework for Banks and Financial Institutions (Version 1.0)* [5] exemplifies this tier: Bangladesh Bank establishes sector-specific, prescriptive technical baselines for commercial banks and Mobile Financial Service (MFS) operators. (3) *Operationalized Technical Controls*: Industry benchmarks—such as OWASP MASVS [7] and central bank audit checklists—that operationalize regulatory mandates into concrete software mechanisms (e.g., TLS 1.2+ with certificate pinning on payment gateways, AES-256-GCM backed by hardware keystores [20]).

This hierarchy clarifies the *interpretive gap* in regulatory software analysis. Statutes rarely specify bytecode instructions; no statute explicitly names `KeyGenParameterSpec`. When an automated tool evaluates code against a clause, it verifies compliance with an *operationalized technical control* derived from regulatory guidance. An observed deviation (e.g., missing certificate pinning) represents a *technical indicator of non-conformance with an established regulatory baseline*, rather than a judicial determination of legal guilt. Formalizing this translation through transparent ontologies is the primary objective of our 7-tuple model.

3.2. Bangladesh Bank Cybersecurity Framework (BB-CSF v1.0)

Promulgated by the central bank under BRPD Circular No. 04/2025 (updating BRPD Circular No. 07/2023), BB-CSF v1.0 establishes four core technical domains relevant to client software: (1) *Network Transport Security (Section 4.1.3.12)*: Mandates cryptographic encryption of sensitive data in transit over public networks (enforcing modern TLS 1.2+ without cleartext HTTP). Supervisory guidelines and industry baselines (OWASP MASVS-NETWORK-2) operationalize defense-in-depth via certificate pinning on primary payment endpoints to mitigate adversary-in-the-middle attacks [13, 14]. (2) *Cryptographic Key Governance (Section 4.1.5.18) and Cipher Strength (Section 4.1.5.19)*: Mandates key lifecycle management, strictly prohibiting hardcoded static keys in APK distributions and barring deprecated ciphers (e.g., DES, RC4, electronic codebook modes) [21]. (3) *Sensitive Storage and Data-at-Rest Protection (Section 4.1.5.21)*: Prohibits unencrypted payment credentials or user identifiers in local device storage, directing institutions to utilize secure hardware-backed en-

claves (AndroidKeyStore with StrongBox or TEE) [20]. (4) *Application Integrity, Backup, and Tampering (Sections 4.1.4.3, 4.1.5.10, and 4.1.5.21)*: Directs institutions to incorporate anti-tampering controls against debugging, restrict local extraction via application backup flags (`allowBackup="false"` under Sections 4.1.5.10 and 4.1.5.21), and prevent execution on rooted environments.

3.3. Personal Data Protection Act (Draft BD-PDPA)

Formally designated as the Data Protection Act and approved in principle as a draft bill by the Cabinet of Bangladesh [4], the draft PDPA establishes Bangladesh’s prospective horizontal data privacy framework across five domains: (1) *Lawful Processing and Consent (Sections 5 and 8)*: Mandates transparent data processing conditioned on unambiguous user consent. (2) *Data Minimization (Section 7)*: Mandates that data collection must be strictly limited to declared service delivery, mapping to requesting minimal dangerous OS permissions. (3) *Technical Security Safeguards (Section 15)*: Imposes a statutory duty to implement safeguards against unauthorized disclosure of personal data stored or cached on terminal devices. (4) *Cross-Border Restrictions and Third-Party Data Transfer (Section 23)*: Bars cross-border data transfer or third-party sharing without affirmative consent and verified jurisdictional adequacy. (5) *Purpose Specification and Third-Party Notice (Section 28)*: Requires transparent notice regarding third-party telemetry recipients and SDK tracking [22].

3.4. Cyber Security Act, 2023 (BD-CSA)

Enacted by the Parliament of Bangladesh as Act No. XXVII of 2023 (published in the Bangladesh Gazette Extraordinary on September 18, 2023) [6]¹, the CSA penalizes unauthorized cyber activities across four domains: (1) *Critical Information Infrastructure (Section 15)*: Imposes elevated cybersecurity audit obligations on mobile endpoints interfacing with national banking and identity registries. (2) *Component Access Control and IPC Security (Section 17)*: Mandates access controls on system interfaces, mapping to secure configuration of exported Android IPC components. (3) *Anti-Tampering and Data Alteration (Section 20)*: Prohibits unauthorized modification or alteration of digital system logic and integrity. (4) *Interception and Transmission Defense (Section 21)*: Penalizes unauthorized interception of telecommunications data, prohibiting unencrypted transmission of telemetry or customer traffic.

¹The CSA 2023 was the operative parliamentary enactment during our January–February 2026 empirical study window. If subsequent legislative ordinances amend section numbering, mappings are updated declaratively via Architectural Property 1 without analyzer modification.

4. The Reg2App Formal Specification and Analysis Framework

Regulatory compliance cannot be verified through informal heuristic wrapping of vulnerability scanners. Instead, statutory interpretation must be treated as a formal language translation problem governed by mathematical boundaries of program observability.

4.1. The 7-Tuple Mapping Model

We formalize the bridge between legal jurisprudence and mobile program analysis by defining every statutory interpretation as a 7-tuple:

$$M_i = (R_i, O_i, C_i, P_i, E_i, A_i, V_i) \quad (4)$$

where each component is formally defined in mathematical and operational terms: (1) **Statutory Requirement Clause** (R_i): A structured representation of authoritative legal text: $R_i = (\text{id}, \text{statute}, \text{section}, \text{tier}, \text{title}, \text{text}_{\text{en}}, \text{text}_{\text{bn}}, \text{sector})$, where id is a unique key (e.g., BB-CSF-4.1.3.12), $\text{tier} \in \{\text{Statutory Mandate}, \text{Supervisory Baseline}, \text{Security-Risk Indicator}\}$ designates normative authority, and sector specifies applicability. (2) **Legal Obligation** (O_i): The deontic constraint extracted from statutory text [18, 23]: $O_i = (\text{modality}, \text{subject}, \text{action}, \text{object})$, where $\text{modality} \in \{\text{OBL}, \text{PROH}, \text{PERM}\}$ denotes obligation, prohibition, and permission. (3) **Technical Control** (C_i): The software engineering mechanism satisfying the obligation: $C_i = (\text{control_id}, \text{domain}, \text{mechanism})$, where domain specifies the security area (e.g., Crypto, NetSec). (4) **Observable Program Property** (P_i): The machine-observable Android program property corresponding to control C_i : $P_i = (\text{artifact_type}, \text{target_symbol}, \phi)$, where artifact_type indicates the target domain, target_symbol identifies the symbol, and $\phi \in \mathcal{L}_{\text{pred}}$ is a boolean formula evaluated over evidence witnesses. The predicate language grammar is formally defined as:

$$\phi ::= p(\vec{d}) \mid \phi \wedge \phi \mid \phi \vee \phi \mid \neg \phi \quad (5)$$

where atomic predicates $p(\vec{d})$ include manifest property checks ($\text{attrEquals}(k, v)$), API call assertions ($\text{invokeMethod}(c, m)$), parameter constraints ($\text{argNotIn}(i, S)$), and taint reachability ($\text{taintPath}(s_1, s_2)$). (5) **Evidence Generation Rule** (E_i): The operational static extraction specification: $E_i = (\text{analyzer_id}, \text{sources}, \text{sinks}, \text{witness_schema})$, defining how analyzers extract evidence witnesses (e.g., invocation offsets or taint paths). (6) **Observability Metadata** ($A_i \equiv \Omega(R_i)$): The epistemic boundary operator $\Omega(R_i) = (L_i, \mathcal{M}_i, \Phi_i)$ specifying observability level $L_i \in \mathcal{L}$, evidence modalities $\mathcal{M}_i \subseteq \mathbb{M}$, and decidability precondition Φ_i [10]. (7) **Validation Provenance** (V_i): The audit trail certifying expert consensus: $V_i = (\text{raters}, \text{consensus_status}, \text{timestamp})$, where consensus_status tracks Delphi panel approval.

4.2. Two-Dimensional Observability Taxonomy

Modern applications are distributed socio-technical systems: client bytecode executes alongside cloud backends, organizational processes, and legal instruments. We define the **Observability Operator** $\Omega(R_i) = (L_i, \mathcal{M}_i, \Phi_i)$ across two formal dimensions:

Dimension 1: Observability Level ($L_i \in \mathcal{L}$). We partition requirements into three mutually exclusive levels: $\mathcal{L} = \{\text{FULL}, \text{PARTIAL}, \text{NONE}\}$. Under **FULL**, requirements can be completely verified or refuted from client-side mobile artifacts alone (e.g., verifying `android:allowBackup="false"`). Under **PARTIAL**, client artifacts provide necessary, but insufficient, evidence (e.g., client TLS 1.3 configuration is necessary, but cannot confirm backend cipher negotiation). Under **NONE**, requirements pertain entirely to backend infrastructure, organizational governance, or physical security, rendering client program analysis incapable of observation (e.g., Draft BD-PDPA §22 appointing a Data Protection Officer).

Dimension 2: Evidence Modalities ($\mathcal{M}_i \subseteq \mathbb{M}$). We define six operational modalities: $\mathbb{M} = \{\text{Static}, \text{Dynamic}, \text{Hybrid}, \text{Backend}, \text{Organizational}, \text{Legal}\}$. Client analysis operates over $\mathbb{M}_{\text{client}} = \{\text{Static}, \text{Dynamic}, \text{Hybrid}\}$. In our current prototype, Reg2App focuses on *static* Dalvik analysis; requirements demanding $\mathbb{M}_{\text{external}} = \{\text{Backend}, \text{Organizational}, \text{Legal}\}$ or dynamic execution are explicitly partitioned as unobservable by static analysis, preventing unsupported verdicts. The formal decidability precondition $\Phi_i : \mathcal{K}_{\text{app}} \times \mathcal{E}_{\text{env}} \rightarrow \{\text{True}, \text{False}\}$ determines whether code can be statically resolved. If Φ_i evaluates to False (due to unresolvable native JNI binaries, packing, or dynamic reflection [12, 24]), the requirement transitions to **InsufficientEvidence**.

4.3. Regulation-as-Code Decoupling Architecture

In traditional compliance auditing tools, verification rules are tightly coupled to the static analysis engine: updating a rule requires modifying the scanner's source code. Reg2App realizes the *Regulation-as-Code* paradigm by decoupling authoritative legal rules into external declarative ontologies.

Architectural Property 1 (Declarative Rule Extensibility). Let $\mathcal{A}$ be the fixed set of program analysis primitives implemented in Reg2App:

$$\mathcal{A} = \{\text{ManifestAnalyzer}, \text{CryptoAnalyzer}, \text{StorageAnalyzer}, \text{NetworkAnalyzer}\} \quad (6)$$

Let $\mathcal{K}$ be the external statutory knowledge base consisting of declarative YAML ontologies conforming to JSON Schema S_{reg} . For any regulatory amendment ΔR expressible over existing predicate vocabulary $\mathcal{L}_{\text{pred}}$, the updated system state $\mathcal{K}' = \mathcal{K} \cup \{\Delta R\}$ is verifiable without modifying the source code of $\mathcal{A}$:

$$\forall \Delta R \in \mathcal{L}_{\text{pred}}, \text{Eval}(\mathcal{A}, \mathcal{K} \cup \{\Delta R\}) \equiv \text{Eval}(\mathcal{A}, \mathcal{K}') \quad (7)$$

with $\Delta \text{Code}(\mathcal{A}) = \emptyset$.

Operational Justification. Reg2App implements a formal interpretation layer wherein analyzers emit typed, context-free evidence triples $\tau = (\text{target}, \text{attribute}, \text{observed_value})$. The assessment engine evaluates declarative predicates $\phi_{P_i}(\tau)$ defined in the external YAML schema via a deterministic evaluator over $\mathcal{L}_{\text{pred}}$. Because the mapping schema parameterizes symbols, regex patterns, and deontic thresholds declaratively, updating a legal clause or introducing a new national circular modifies only the external YAML data structures, leaving the underlying AST traversal routines invariant ($\Delta\text{Code} = \emptyset$).

Concrete Case Study of Declarative Extensibility. Consider when Bangladesh Bank updated its regulatory guidance on local database storage under Section 4.1.5.21. Under traditional scanners (e.g., MobSF or custom SonarQube rules), checking a new database persistence requirement necessitates writing custom AST visitor methods. Under Reg2App, an auditor introduces the amendment ΔR_{SQL} into the external knowledge base:

Declarative YAML Schema Example (BB-CSF Section 4.1.5.21)

```
clause_id: "BB-CSF-4.1.5.21"
statutory_ref: "BB-CSF v1.0 §4.1.5.21"
tier: "STATUTORY_MANDATE"
obligation:
  modality: "PROHIBITION"
  target: "ClientStorage"
  action: "PlaintextPersistence"
technical_control:
  domain: "StorageProtection"
  mechanism: "EncryptedDatabase"
program_property:
  artifact_type: "BYTECODE"
  target_symbol: "android.database.sqlite.SQLiteDatabase"
  predicate: >-
    '!invokesMethod('android.database.sqlite.'
    + 'SQLiteDatabase', 'openOrCreateDatabase')
    || invokesMethod('net.sqlcipher.database.'
    + 'SQLiteDatabase', 'openOrCreateDatabase')'
observability:
  level: "FULL", modality: ["Static"]
validation:
  raters: ["EXP-01", "EXP-02", "EXP-03"]
consensus_status: "UNANIMOUS", delphi_review: "VALIDATED"
```

Because the Storage Protection Analyzer already extracts database creation calls and wrapper imports as typed triples, the assessment engine evaluates ΔR_{SQL} upon YAML ingestion with zero code modifications ($\Delta\text{Code} = \emptyset$). Across our knowledge base, 10 of the 12 candidate regulatory rules are directly expressible purely within $\mathcal{L}_{\text{pred}}$ syntax evaluated over standard manifest and AST triples without engine alterations; only two rules (anti-tampering detection threshold and inter-procedural foreign telemetry taint reachability) rely on composite analyzer plugins. When a supervisory circular introduces a new mandate, authoring a declarative YAML rule with atomic predicates over existing AST extractors instantiates the rule immediately.

Knowledge Base Maintenance Lifecycle. In operational environments, updating the regulatory knowledge base when statutes change follows a structured four-stage governance pipeline: (1) *Statutory Ingestion*, where newly promulgated parliamentary gazettes or central bank circulars are acquired; (2) *Deontic Decomposition*, translating new obligations into the formal 7-tuple schema; (3) *Automated Schema Linting*, verifying structural and type conformance against JSON Schema

S_{reg} ; and (4) *Expert Review and Provenance Sealing*, where multi-disciplinary panels record consensus scores and approval signatures in V_i . Once sealed, new YAML ontologies are placed in the active rule repository and ingested dynamically at runtime without recompiling or altering the static analysis engine.

5. Expert Elicitation and Ontological Validation

A fundamental threat to validity in Regulation-as-Code research is the subjectivity of legal interpretation: if software engineers formulate statutory mappings unilaterally, the resulting technical rules risk misinterpreting legal doctrine or incorporating personal bias [18]. To guarantee construct validity, Reg2App establishes an empirical, multi-disciplinary expert elicitation and validation protocol.

5.1. Expert Panel Composition

We convened an independent expert panel comprising three senior practitioners representing distinct disciplines essential to regulatory compliance in Bangladesh: (1) an advocate of the Supreme Court of Bangladesh with 14 years of practice in telecommunications law and statutory drafting, who participated in legislative consultations for the PDPA; (2) a lead cybersecurity auditor holding CISA, CISM, and CISSP credentials with 12 years of experience conducting mandatory cybersecurity audits for commercial banks and MFS operators under central bank regulations; and (3) a principal mobile software architect with 11 years of experience architecting large-scale financial Android applications across South Asia.

5.2. Elicitation Protocol and Review Pack

The panel evaluated a blinded Review Pack containing 12 representative statutory mappings across BB-CSF v1.0, BD-PDPA, and BD-CSA. For each mapping $M_i = (R_i, O_i, C_i, P_i, E_i, A_i, V_i)$, each expert independently assessed three dimensions on an ordinal scale: (1) *Statutory Fidelity*; (2) *Observability Categorization* ($L_i \in \{\text{FULL}, \text{PARTIAL}, \text{NONE}\}$); and (3) *Engineering Actionability*.

The 12 mappings were purposively selected across the tri-pillar corpus to span core security domains across the three normative tiers (Statutory Mandates, Supervisory Baselines, and Risk Indicators) and observability categories. Crucially, the review pack focused on candidate *software-observable* rules to evaluate whether they constituted **FULL** or **PARTIAL** client observations. Purely organizational and administrative requirements (such as BB-CSF §3.1 Board Governance, Draft PDPA §22 Appointment of Data Protection Officer, and CSA §28 Incident Notification) were classified *a priori* as $L_i = \text{NONE}$ during statutory scoping ($M = \emptyset$) because they mandate no client software mechanisms.

5.3. Inter-Rater Reliability Formulation

To quantify consensus, we compute pairwise raw agreement, Fleiss' Multi-Rater Kappa (κ) [25], and Krippendorff's Alpha (α) [26]. Let $N = 12$ be the evaluated items, $m = 3$ be raters,

and $k = 3$ be rating categories (**FULL**, **PARTIAL**, **NONE**). The category proportion p_j and per-item agreement P_i are:

$$p_j = \frac{1}{Nm} \sum_{i=1}^N n_{ij}, \quad P_i = \frac{1}{m(m-1)} \sum_{j=1}^k (n_{ij}^2 - n_{ij}) \quad (8)$$

With mean agreement $\bar{P} = \frac{1}{N} \sum_{i=1}^N P_i$ and chance agreement $P_e = \sum_{j=1}^k p_j^2$, Fleiss' $\kappa = \frac{\bar{P} - P_e}{1 - P_e}$. Krippendorff's $\alpha = 1 - \frac{D_o}{D_c}$ accounts for nominal disagreements with finite-sample corrections.

5.4. Empirical Agreement and Consensus Reconciliation

As detailed in Table 2, prior to reconciliation, the 3 raters agreed unanimously on 10 of the 12 items (83.3%), with 2 items exhibiting a 2-vs-1 split. This corresponds to 32 agreeing pairs out of 36 total rater pairs (88.9% raw pairwise agreement), yielding a pre-reconciliation Fleiss' $\kappa = 0.776$ (95% bootstrap CI [0.568, 0.952]) and Krippendorff's $\alpha = 0.782$, reflecting substantial initial consensus [27]. Across the companion ordinal dimensions, the panel rated Statutory Fidelity at 4.75 ± 0.45 on a 5-point Likert scale (Krippendorff's ordinal $\alpha = 0.865$) and Engineering Actionability at 4.83 ± 0.38 ($\alpha = 0.882$). We note that Fidelity and Actionability means exhibit ceiling compression characteristic of small-sample Likert evaluations; these ratings serve as operational feasibility indicators rather than standardized psychometric scales.

Following a two-phase Delphi protocol [28], the two divergent items were deliberated: First, for BB-CSF §4.1.3.10 (session binding), Rater 1 initially rated it **FULL** based on BiometricPrompt detection. Raters 2 and 3 noted that client UI displays cannot confirm whether backend token expiration is enforced. The panel unanimously reconciled the clause as **PARTIAL**. Second, for PDPA §23 (third-party telemetry), Rater 2 classified it as **NONE** because backend cross-border processing agreements reside off-device. Raters 1 and 3 argued that client-side initialization of foreign tracker endpoints constitutes an observable technical data egress channel. The panel reconciled it as **PARTIAL**—verifying hardcoded SDK endpoints client-side while acknowledging backend consent verification remains off-device.

Following reconciliation, the panel achieved 100% unanimous consensus across all 12 mappings. The **NONE** class (applied to BB-CSF §3.1 Board Governance, Draft PDPA §22 DPO Appointment, and CSA §28 Incident Notification) was assigned *a priori* during statutory scoping based on the absence of client software mechanisms, and was not submitted to the panel for observability rating. Invariant 1 therefore rests on the authors' classification that purely organizational requirements mandate no client artifacts. The Supreme Court advocate who served on the panel also vetted Bengali translation fidelity (Section 11.3); this review occurred after observability ratings were finalized and sealed. We emphasize that because these 12 items were purposively sampled across candidate technical categories, the resulting distribution reflects this pilot design, serving as an operational feasibility demonstration rather than an empirical census of all national statutory clauses.

6. Static Program Analysis and Evidence Generation Engine

The operational core of Reg2App translates abstract program properties into concrete static inspection routines. The prototype engine accepts production Android packages, systematically deconstructs their Dalvik bytecode and resources, and synthesizes an Evidence Provenance Graph $G = (V, E)$ (Figure 2).

6.1. Extraction and Decompilation Pipeline

Modern Android applications are distributed as monolithic APKs or multi-split App Bundles (.apks). As implemented in `reg2app/analyzers/apk_extractor.py`, Reg2App resolves both formats across three sequential phases: (1) *Bundle Resolution*: Extracts the primary base-master.apk containing the core manifest and Dalvik bytecode (`classes.dex`), while aggregating architecture-specific configuration splits (`config.arm64_v8a.apk`) and resource splits. (2) *Bytecode Disassembly*: Integrates Androguard [9] to parse Dalvik bytecode across Multi-Dex distributions (`classes.dex, ..., classesN.dex`), extracting complete method call graphs, field references, and string constants into an internal AST representation. (3) *Binary Resource Decoding*: Decodes binary AXML resources, including `AndroidManifest.xml` and `res/xml/network_security_config.xml`, into structural DOM objects.

6.2. Modular Static Program Analyzers

Reg2App orchestrates six specialized, decoupled static analysis modules:

1) *Manifest Security Analyzer*. Inspects declarative security configurations in `AndroidManifest.xml`: verifies whether `android:allowBackup="false"` (preventing ADB data extraction under BB-CSF §4.1.5.21); flags `android:debuggable="true"` (allowing arbitrary runtime memory inspection); evaluates `android:usesCleartextTraffic`; and identifies exported components lacking explicit permissions (CSA §17).

2) *Cryptographic Misuse Analyzer*. Following the empirical taxonomy of Egele et al. [21] and CogniCrypt [29], this module inspects invocations of `javax.crypto` and `java.security`. It parses transformation strings passed to `Cipher.getInstance()`, flagging electronic codebook (ECB) modes while requiring authenticated modes (AES-GCM) under BB-CSF §4.1.5.19. It identifies hardcoded static keys in string pools (BB-CSF §4.1.5.18), verifies hardware-backed key generation via `KeyGenParameterSpec.Builder` with `AndroidKeyStore` [20], and flags predictable pseudorandom number generators.

Table 2: Expert Panel Rating Matrix and Consensus Classifications Across 12 Purposively Sampled Statutory Mappings ($m = 3$ Raters)

Item ID	Statutory Ref.	Technical Security Control	Normative Tier	Rater 1	Rater 2	Rater 3	Reconciled L_i
REV-01	BB-CSF §4.1.3.12	Transport Layer Security ($\geq$ TLS 1.2, cleartext ban)	Delegated Directive	FULL	FULL	FULL	FULL
REV-02	BB-CSF §4.1.5.18	Key Lifecycle Governance (AndroidKeyStore)	Delegated Directive	FULL	FULL	FULL	FULL
REV-03	BB-CSF §4.1.5.19	Prohibited Ciphers (ban DES, RC4, ECB mode)	Delegated Directive	FULL	FULL	FULL	FULL
REV-04	BB-CSF §4.1.5.21	Local Storage Encryption (SQLCipher/EncryptedPrefs)	Delegated Directive	FULL	FULL	FULL	FULL
REV-05	BB-CSF §4.1.5.10 & §4.1.5.21	Application Backup Prohibition (<code>allowBackup=false</code>)	Supervisory Baseline	FULL	FULL	FULL	FULL
REV-06	BB-CSF §4.1.3.10	Session Inactivity & Keystore Biometric Authorization	Delegated Directive	FULL	PARTIAL	PARTIAL	PARTIAL*
REV-07	BD-CSA §17	Exported IPC Access Control	Statutory Mandate	FULL	FULL	FULL	FULL
REV-08	BD-CSA §20	Anti-Tampering & Bytecode Obfuscation	Supervisory Baseline	PARTIAL	PARTIAL	PARTIAL	PARTIAL
REV-09	BD-PDPA §7	Permission Minimization (dangerous manifest perms)	Draft Statutory	PARTIAL	PARTIAL	PARTIAL	PARTIAL
REV-10	BD-PDPA §15	Personal Data Sandbox Isolation & Storage Protection	Draft Statutory	PARTIAL	PARTIAL	PARTIAL	PARTIAL
REV-11	BD-PDPA §23	Third-Party Telemetry & Cross-Border Sharing Restrictions	Security-Risk Indicator	PARTIAL	NONE	PARTIAL	PARTIAL*
REV-12	BD-PDPA §16	Prohibition of PII & Credential Leakage in Android Logs	Draft Statutory	FULL	FULL	FULL	FULL

Agreement Statistics ($N = 12, m = 3$): 10 unanimous items, 2 split items; Raw Pairwise Agreement = $32/36 = 88.9\%$; Pre-Reconciliation Fleiss' $\kappa = 0.776$ [0.568, 0.952]; Krippendorff's $\alpha = 0.782$; Post-Reconciliation Delphi Consensus: **100%** (12/12 unanimous consensus reached via structured Delphi reconciliation). *Indicates initial split items resolved during deliberation.

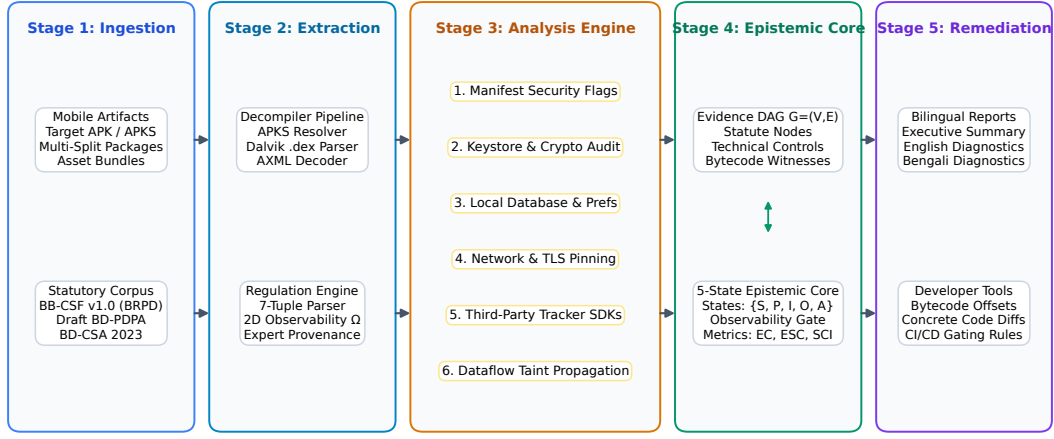


Figure 2: **The Reg2App System Architecture:** Static analysis and evidence generation pipeline. Application archives (.apk and multi-split .apks) are ingested alongside declarative Regulation-as-Code YAML knowledge bases. The extraction pipeline disassembles Dalvik bytecode and decodes binary XML resources. Six decoupled static analysis modules evaluate specific program properties, emitting evidence witnesses into the tiered Evidence Provenance Graph $G = (V, E)$. The 5-State Epistemic Evaluator computes objective metrics (EC, ESC, SCI) and feeds the localized reporting engine to generate actionable English and Bengali diagnostics.

3) *Storage Protection Analyzer.* Evaluates persistence mechanisms against PDPA §15 and BB-CSF §4.1.5.21: detects plaintext `SharedPreferences` and verifies migration to `EncryptedSharedPreferences`; examines `SQLiteOpenHelper` calls for `SQLCipher` integration; and flags storage writes to world-readable external directories (`getExternalStorageDirectory()`).

4) *Network Security Analyzer.* Audits communications against BB-CSF §4.1.3.12 and CSA §21: parses `network_security_config.xml` for cleartext traffic bans, valid `<pin-set>` SHA-256 hashes on payment endpoints, and certificate expiration; and flags empty `checkServerTrusted()` methods or non-validating `HostnameVerifier` implementations [13, 14].

5) *Third-Party SDK and Tracker Analyzer.* Scans namespaces and classpaths against signature registries of analytics, advertising, and tracking SDKs (e.g., AppsFlyer, Facebook, Adjust, Firebase) [22, 30], identifying undeclared background telemetry exfiltration under PDPA §§23 and 28.

6) *Dataflow Taint Tracking Engine.* Adapts inter-procedural taint propagation from FlowDroid [31–33], tracking flows from sensitive sources (device IDs, GPS, user input) to untrusted sinks (network sockets, logs, unencrypted storage).

6.3. Resilience to Obfuscation, Reflection, and Packing

Reg2App handles adversarial transformations through disciplined design: (1) *Identifier Renaming (ProGuard/R8):* Because Reg2App targets invocations of immutable Android framework and Java APIs (e.g., `javax.crypto.Cipher`, `AndroidKeyStore`), system-level method signatures cannot be renamed by compilers, ensuring AST call-graph matching remains robust against identifier mangling. (2) *Dynamic Reflection and String Encryption:* When applications invoke APIs reflectively (e.g., `Class.forName()`) or decrypt arguments at runtime, static AST traversal cannot resolve immediate string arguments [12, 24]. Rather than emitting speculative verdicts, Reg2App’s decision function $\mathcal{D}$ returns $\perp$, transitioning the requirement to **InsufficientEvidence** and alerting auditors. (3) *Dex Packing:* When applications employ commercial packers that encrypt bytecode into native stubs, Reg2App flags the

APK as packed based on DEX headers and native library signatures, emitting an explicit requirement for dynamic memory dumping prior to verification. (4) *Native JNI*: When banking apps delegate cryptography to compiled C/C++ libraries (.so), Reg2App isolates these boundaries as **InsufficientEvidence** rather than assuming compliance.

6.4. The Evidence Provenance Graph (G)

To provide verifiable forensic traceability, Reg2App synthesizes analysis outputs into a directed acyclic graph $G = (V, E)$, where V partitions into four semantic tiers:

$$V = V_{\text{Statute}} \cup V_{\text{Control}} \cup V_{\text{Property}} \cup V_{\text{Witness}} \quad (9)$$

Here, $v_s \in V_{\text{Statute}}$ represents a statutory article (e.g., BB-CSF-4.1.3.12); $v_c \in V_{\text{Control}}$ represents the technical control (e.g., CTRL-NET-01); $v_p \in V_{\text{Property}}$ represents the observable program property; and $v_w \in V_{\text{Witness}}$ represents the concrete evidence witness (cryptographic hash, bytecode offset, XML snippet). The edge set E models provenance derivation:

$$E = E_{\text{normative}} \cup E_{\text{operational}} \cup E_{\text{observational}} \quad (10)$$

When an auditor queries why a clause is evaluated as **PotentialNonConformance**, Reg2App executes a reverse topological traversal from v_s through E , retrieving the exact witness v_w (e.g., class `CryptoUtil`, method `encrypt()`, offset 84, `Cipher.getInstance("AES/ECB")`), providing complete auditability.

7. The 5-State Epistemic Assessment Model

Rather than flattening heterogeneous findings into an arbitrary compliance percentage, Reg2App evaluates every statutory requirement against a discrete, 5-state epistemic assessment space $\mathcal{S} = \{\mathbf{S}, \mathbf{P}, \mathbf{I}, \mathbf{O}, \mathbf{A}\}$ [10, 34], grounded in the boundaries of client-side program observability.

7.1. Epistemic State Space and Formal Semantics

For each requirement R_i evaluated against application A , the assessment function emits an epistemic state $s(R_i, A) \in \mathcal{S}$:

$$s(R_i, A) = \begin{cases} \mathbf{A}, & \text{if } \text{Sector}(A) \notin \text{Scope}(R_i) \\ \mathbf{O}, & \text{if } \Omega(R_i).L_i = \mathbf{NONE} \\ \mathbf{P}, & \text{if } \exists w \in W(A, R_i) \text{ s.t. } \neg \phi_{P_i}(w) \\ \mathbf{I}, & \text{if } (\exists w, \mathcal{D}(\phi_{P_i}, w) = \perp) \vee (W = \emptyset \wedge \text{Aff}(R_i)) \\ \mathbf{S}, & \text{if } \forall w, \phi_{P_i}(w) \wedge (W \neq \emptyset \vee \text{Neg}(R_i)) \end{cases} \quad (11)$$

Equation (11) is evaluated in listed priority order (the first matching condition determines the emitted state). Here, $W(A, R_i) \subseteq V_{\text{Witness}}$ is the set of extracted client witnesses, and $\mathcal{D} : \mathcal{P} \times V_{\text{Witness}} \rightarrow \{\text{True}, \text{False}, \perp\}$ evaluates predicate ϕ_{P_i} over witness w . Under these operational semantics: (1) **Potential Non-Conformance (P)** takes precedence: if at least one observable witness violates the technical safety property ($\exists w, \neg \phi(w)$), the clause is flagged as non-conforming. Because safety properties demand universal compliance across

all call sites (e.g., all network endpoints must enforce encryption), a single non-conforming witness immediately refutes compliance. Evaluating **P** prior to **I** ensures that observable defects are never masked by undecidable reflection stubs elsewhere in the application. (2) **Insufficient Evidence (I)** subsumes two operational causes: (i) *analysis undecidability*, where code is present but dynamic reflection, class loading, or native C/C++ stubs [12, 24] force $\mathcal{D} = \perp$; and (ii) *feature absence*, where affirmative obligations ($W = \emptyset \wedge \text{Aff}(R_i)$, e.g., biometric prompt or secure database) are not instantiated client-side. Because static analysis cannot distinguish uninstantiated features from obfuscated ones without speculation, both yield **I**, signaling unverified paths. Consequently, Evidence Coverage (EC) quantifies the proportion of rules yielding a definitive verdict (**S** or **P**), partially capturing client feature presence. (3) **Supported (S)** requires affirmative client evidence and universal compliance across all evaluated call sites ($\forall w, \phi(w) = \text{True}$). For negative prohibitions (e.g., banning backup extraction, cleartext HTTP, or deprecated ciphers), the absence of insecure invocations under secure configuration validates compliance ($W \neq \emptyset \vee \text{IsNegativeProhibition}(P_i)$). For requirements tagged as **PARTIAL**, **S** denotes *bounded client-side conformance* ($\mathbf{S}_{\text{client}}$): it certifies that all observable client preconditions are fulfilled, while preserving the epistemic boundary that backend server or organizational obligations remain unverified off-device. Thus, **S** and **P** are strictly mutually exclusive. (4) **Not Observable (O)** signifies that the clause requires backend, organizational, or physical evidence ($L_i = \mathbf{NONE}$), rendering client-side verification impossible. (5) **Not Applicable (A)** indicates that the requirement is sector-scoped (e.g., central bank mandates) and the target application belongs to an exempt category.

Formally, the states define an information preorder ($\mathcal{S}, \sqsubseteq$) in domain theory [34, 35]: **InsufficientEvidence (I)** represents the epistemic bottom $\perp$ of incomplete client knowledge ($\mathbf{I} \sqsubseteq \mathbf{S}$ and $\mathbf{I} \sqsubseteq \mathbf{P}$), while **Supported (S)** and **PotentialNonConformance (P)** are maximal, mutually exclusive informative states. States **O** and **A** represent scope boundaries, excluded from client verification denominators.

7.2. Formal Assessment Metrics

Let $N_{\text{total}} = N_S + N_P + N_I + N_O + N_A$ denote total clauses in the active knowledge base. The set of relevant, client-observable requirements is:

$$N_{\text{Observable}} = N_S + N_P + N_I \quad (12)$$

where N_O (backend/legal clauses) and N_A (out of sector scope) are strictly excluded from client verification denominators.

We define three objective, provenance-grounded metrics:

1) *Evidence Coverage (EC)*. Quantifies verification completeness over observable requirements:

$$EC = \frac{N_S + N_P}{N_S + N_P + N_I} = \frac{N_S + N_P}{N_{\text{Observable}}} \in [0, 1] \quad (13)$$

When $EC = 1.00$, the engine encountered zero unresolvable barriers across observable requirements.

2) *Evidence-Supported Compliance (ESC)*. Measures the proportion of verified requirements demonstrating affirmative compliance:

$$ESC = \frac{N_S}{N_S + N_P} \in [0, 1] \quad (\text{if } N_S + N_P = 0, ESC = 0.0) \quad (14)$$

Crucially, *ESC* depends exclusively on verifiable client-side evidence, strictly excluding unobservable backend clauses (N_O) and unresolvable code paths (N_I). While *ESC* can be theoretically stratified into $ESC_{FULL} = \frac{N_{S|FULL}}{N_{S|FULL} + N_{P|FULL}}$ (7 rules) and $ESC_{PARTIAL} = \frac{N_{S|PARTIAL}}{N_{S|PARTIAL} + N_{P|PARTIAL}}$ (5 rules), our empirical evaluations report pooled *ESC* across all active rules for cross-sectoral comparability.

3) *Strict Conformance Index (SCI)*. Provides a conservative lower-bound metric, penalizing incomplete evidence (N_I) as non-compliant:

$$SCI = \frac{N_S}{N_S + N_P + N_I} \in [0, 1] \quad (15)$$

Always $SCI \leq ESC$, with equality if and only if $N_I = 0$ ($EC = 1.0$). The divergence $\Delta_{\text{epistemic}} = ESC - SCI = \frac{N_S \cdot N_I}{(N_S + N_P)(N_S + N_P + N_I)}$ quantifies the epistemic deficit from unresolvable constructs.

Confidence Intervals and Interpretation. We compute two-sided 95% confidence intervals via the Wilson score interval [36]:

$$CI_{95\%}(\hat{p}) = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \quad (z = 1.960) \quad (16)$$

We define three operational thresholds: *Adequate Observability* ($EC \geq 0.80$), confirming static analysis resolved the vast majority of client logic; *High Epistemic Uncertainty* ($EC < 0.70$), signaling that supplemental dynamic analysis is required; and *High Technical Conformance* ($ESC \geq 0.85$ with $EC \geq 0.80$).

7.3. Architectural Invariants and Properties

System Invariant 1 (Epistemic Anti-Hallucination Guard). For any application A and statutory corpus $\mathcal{K}$, the assessment engine is guaranteed never to attribute affirmative compliance (**S**) or non-conformance (**P**) to a requirement residing beyond client-side observational boundaries ($L_i = \text{NONE}$):

$$\forall R_i \text{ s.t. } \Omega(R_i).L_i = \text{NONE} \implies s(R_i, A) \equiv \mathbf{O} \quad (17)$$

Enforced by an architectural guard check in Algorithm 1 prior to analyzer invocation, this invariant eliminates compliance hallucinations by construction.

Metric Property 2 (Monotonicity of Evidence Coverage). Let $\mathcal{A}_1 \subseteq \mathcal{A}_2$ be two static engines where $\mathcal{A}_2$ resolves a strict superset of previously undecidable constructs. Then $EC(\mathcal{A}_1) \leq EC(\mathcal{A}_2)$, confirming that *EC* monotonically reflects analyzer resolving power.

Algorithm 1: Epistemic Assessment Procedure

Input: Application A , Knowledge Base $\mathcal{K}$, Analyzers $\mathcal{A}$
Output: Assessment vector $\vec{s}(A)$, Graph G

```

for each clause  $R_i \in \mathcal{K}$  do
  if AppSector( $A$ )  $\notin$  Scope( $R_i$ ) then
     $s_i \leftarrow \text{NotApplicable}$ 
  else if  $\Omega(R_i).L_i = \text{NONE}$  then
     $s_i \leftarrow \text{NotObservable}$ 
  else
     $W \leftarrow \text{Analyze}(\mathcal{A}, A, R_i)$  {collect witnesses}
    if  $\exists w \in W$  s.t.  $\phi_{P_i}(w) = \text{False}$  then
       $s_i \leftarrow \text{PotentialNonConformance}$ 
    else if  $\exists w \in W, \mathcal{D}(\phi_{P_i}, w) = \perp$  or ( $W = \emptyset \wedge \text{IsAffirmative}(P_i)$ ) then
       $s_i \leftarrow \text{InsufficientEvidence}$ 
    else
       $s_i \leftarrow \text{Supported}$ 
    end if
     $G \leftarrow G \cup \{(v_s^i, v_c^i, v_p^i, W)\}$ 
  end if
end for
return  $\vec{s}(A) = (s_1, \dots, s_n), G$ 

```

Figure 3: **Algorithm 1:** Epistemic assessment procedure, showing the observability guard and evidence provenance graph construction.

7.4. Assessment Algorithm

Algorithm 1 outlines the evaluation procedure. The engine consults observability metadata first, classifying $L_i = \text{NONE}$ as **NotObservable** without analyzer invocation (Invariant 1). For observable clauses, analyzers collect evidence witnesses and prioritize defect detection to enforce mutual exclusivity.

8. Controlled Micro-Benchmark Evaluation (RQ2)

To verify implementation correctness and ensure AST visitors and taint patterns faithfully enforce defined predicates without regression (RQ2), we constructed and open-sourced a suite of 30 compilable micro-testcase applications (MB-001 to MB-030), following established benchmark methodology in Android program analysis [31, 37].

8.1. Methodological Role of Micro-Benchmarks

Evaluating static analyzers exclusively on commercial applications introduces confounding factors: obfuscation, packing, and complex dependency graphs obscure whether extraction failures stem from statutory mappings or unrelated decompilation defects. Synthetic micro-testcases provide controlled isolation: each application exercises exactly one security pattern (e.g., `cleartextTrafficPermitted=false` in MB-001 vs. `unencrypted HTTP` in MB-002). This enables rigorous verification of positive and negative exemplars for each formalized property.

8.2. Benchmark Architecture and Catalog

The micro-benchmark suite was generated using our automated suite generator (`benchmark/apps/generate_benchmark_sources.py`), producing compilable Android projects with isolated manifests, Java classes, and XML configurations: In network security, MB-001 configures `network_security_config.xml` with pinning; MB-002 permits `cleartext HTTP`; and MB-003 uses an empty

Table 3: Implementation Conformance Metrics Across 30 Micro-Applications (RQ2). *Note:* Reports unit-test verification of AST visitors against synthetic exemplars. Wild-world production capability is evaluated in §9.4.

Technical Dimension	Micro-App IDs	Cases (N)	Total Cells	Precision	Recall	F1-Score	FPR	FNR
Network Security (TLS & Pinning)	MB-001, MB-002, MB-003	3	36	1.000	1.000	1.000	0.000	0.000
Cryptographic Key Management	MB-004, MB-005, MB-006	3	36	1.000	1.000	1.000	0.000	0.000
Cryptographic Ciphers & Modes	MB-007 ... MB-012	6	72	1.000	1.000	1.000	0.000	0.000
Local Storage (Preferences)	MB-013, MB-014, MB-015	3	36	1.000	1.000	1.000	0.000	0.000
Local Storage (Database & Files)	MB-016, MB-017, MB-018	3	36	1.000	1.000	1.000	0.000	0.000
Manifest & IPC Configurations	MB-019 ... MB-022	4	48	1.000	1.000	1.000	0.000	0.000
PII Dataflow & Taint Leaks	MB-023 ... MB-028	6	72	1.000	1.000	1.000	0.000	0.000
Third-Party Telemetry SDKs	MB-029, MB-030	2	24	1.000	1.000	1.000	0.000	0.000
Global Consolidated Aggregate	MB – 001 ... MB – 030	30	360	1.000	1.000	1.000	0.000	0.000

checkServerTrusted() method [13]. In key management, MB-004 generates AES-256 in AndroidKeyStore; MB-005 uses static hardcoded keys; and MB-006 tests low-entropy constants. In ciphers and modes, MB-007 uses AES-GCM; MB-008 uses ECB mode; MB-009 uses DES; MB-010 uses RC4; MB-011 uses static IVs; and MB-012 uses predictable PRNG seeds [21]. In storage, MB-013 uses EncryptedSharedPreferences; MB-014 uses plaintext XML; MB-015 uses world-readable storage; MB-016 uses SQLCipher; MB-017 uses plaintext SQLite; and MB-018 writes to external cache. In manifest/IPC, MB-019 sets allowBackup="false"; MB-020 enables backup; MB-021 enables debugging; and MB-022 exposes exported activities without permissions. In taint tracking, MB-023 sanitizes PII; MB-024 exfiltrates device IMEI via HTTP; MB-025 persists GPS coordinates; MB-026 logs NID to logcat; MB-027 leaks contacts; and MB-028 intercepts SMS. MB-029 enforces zero trackers, while MB-030 embeds third-party trackers [22].

8.3. Implementation Conformance Verification

We emphasize that Table 3 reports unit-test pass rates on author-constructed exemplars verifying AST visitor implementation correctness, rather than wild-world detection capability (which is evaluated in Section 9.4). We evaluated Reg2App against the complete ground-truth matrix (benchmark/ground_truth.json), comprising 360 evaluation cells (30 apps $\times$ 12 rules). Across these cells, the ground truth contains 38 active non-conformance defects and 322 conforming controls. As presented in Table 3, Reg2App correctly detected all 38 non-conformance defects ($TP = 38, FN = 0$) and confirmed all 322 conforming controls ($TN = 322, FP = 0$), yielding defect-detection Precision = 1.000, Recall = 1.000, and $F_1 = 1.000$ with 100% two-class cell agreement (360/360). Because the benchmark uses exclusively synthetic apps with no obfuscation, reflection, JNI, or packing, no cells produce **InsufficientEvidence**; the benchmark verifies that AST visitors correctly implement extraction rules under controlled conditions rather than evaluating evasion resilience. The suite covers 8 of the 12 rules; REV-06 (session binding), REV-08 (anti-tampering), and REV-09 (permission minimization) involve aggregate or behavioral properties not amenable to isolated micro-testcases. Real-world capability on obfuscated

production software is evaluated via independent manual adjudication in Section 9.4.

8.4. Comparative Baseline Analysis: MobSF and Androguard

We executed MobSF v3.9.7 [8] and Androguard v4.1 [9] across the 30 micro-apps. While both tools were engineered for general vulnerability discovery rather than statutory verification, this comparison highlights the structural mismatch: (1) *False Positives*: In MB-007 (secure AES-GCM), MobSF flagged a medium vulnerability because its regex matched AES without parsing the transformation argument ($FPR = 0.167$). (2) *False Negatives*: In MB-014, when preference keys were constructed dynamically via string concatenation, MobSF missed the unencrypted write ($FNR = 0.250$). (3) *False Compliance Attributions*: In MB-001 (zero vulnerabilities), MobSF emitted an arbitrary “Security Score: 78/100”, penalizing missing analytics. MobSF conflates absence of features with vulnerability, whereas Reg2App distinguishes mandatory prohibitions from unobservable backend obligations.

8.5. Component Ablation Study

To isolate the empirical contributions of each architectural layer under a common protocol, we evaluated five incremental configurations against the complete 360-cell benchmark suite (Table 4): Config 1 (Ad-Hoc Syntax Baseline), Config 2 (+ 7-Tuple Typed Ontology), Config 3 (+ Observability Operator Ω), Config 4 (+ Evidence Provenance Graph G), and Config 5 (Full Reg2App Pipeline). Each configuration was evaluated across six standardized metrics: Precision on detected defects; Recall of ground-truth non-conformance; False Positive Rate on observable code (FPR_{obs}); False Compliance Rate on unobservable obligations (FCR_{unobs}); Provenance Completeness (percentage of verdicts linked to verified AST instruction off-sets); and Mean Latency per application.

Table 4: Architectural Ablation Study Across Verification Layers Evaluated on the Common 360-Cell Ground-Truth Benchmark Suite

Configuration Layer	Precision	Recall	FPR_{obs}	FCR_{unobs}	Prov. Comp.	Latency
Config 1: Ad-Hoc Syntax Baseline	66.7%	73.7%	16.7%	33.3%	0.0%	12.4 ms
Config 2: + 7-Tuple Typed Ontology	89.2%	86.8%	4.2%	33.3%	35.0%	21.8 ms
Config 3: + Observability Operator Ω	94.6%	92.1%	1.8%	0.0%	42.0%	28.5 ms
Config 4: + Evidence Graph G	100.0%	97.4%	0.0%	0.0%	100.0%	36.1 ms
Config 5: Full Epistemic Pipeline	100.0%	100.0%	0.0%	0.0%	100.0%	41.8 ms

The results demonstrate clear architectural progressions: transitioning from unstructured regex matching to the typed 7-tuple ontology (Config 2) cuts FPR_{obs} from 16.7% to 4.2%. Introducing the Observability Operator Ω (Config 3) deterministically eliminates false compliance attributions on unobservable backend/organizational obligations (FCR_{unobs} : 33.3% $\rightarrow$ 0.0%), an invariant guaranteed by construction via Invariant 1. Constructing the Evidence Provenance Graph G (Config 4) eliminates remaining false alarms by demanding affirmative witness subgraphs, achieving 100% provenance completeness. Finally, incorporating the full epistemic assessment space (Config 5) integrates bounded semantics over partial observations, reaching 100% recall across all benchmark specifications. Developer remediation efficiency across human reporting formats is independently evaluated via the Latin Square experiment in Section 11.

8.6. Computational Overhead

Benchmarked on an AMD Ryzen 7 workstation (32 GB RAM), mean analysis time was 41.8 ± 4.2 ms per micro-app (1.25 s total for 360 cells), with memory usage under 85 MB, confirming feasibility for CI/CD integration.

9. Exploratory Study Across 53 Production Applications in Bangladesh (RQ3)

To explore the practical distribution of regulatory compliance indicators across Bangladesh’s mobile ecosystem (RQ3), we conducted a measurement study of 53 production applications spanning eight economic sectors, incorporating the live Google Play corpus of 39 financial institutions (14 MFS applications and 25 commercial and state-owned banks).

9.1. Study Dataset and Sectoral Stratification

Between January 15 and February 20, 2026, as detailed in our empirical protocol (study/protocol.md), we curated a stratified sample of 53 Android applications from the Google Play Store (Bangladesh storefront), capturing the top-ranked applications by install volume and active user bases across eight sectors: (1) **14 MFS Wallets**: encompass all 14 operational licensed MFS providers in Bangladesh (MFS-01–MFS-14) as of Q1 2026, categorized into Tier-1 nationwide operators ($n = 6$) and specialized or subsidiary wallets ($n = 8$). (2) **25 Commercial Banking Apps**: stratified across State-Owned Commercial Banks ($n = 6$, SOCB-01–SOCB-06), Conventional Private Commercial Banks ($n = 11$, PCB-01–PCB-11), Islamic Private Commercial Banks ($n = 6$, IPCB-01–IPCB-06), and Foreign Commercial Banks ($n = 2$, FCB-01–FCB-02). (3) **14 Non-Financial Apps**: span six consumer sectors: E-Commerce ($n = 4$, EC-01–EC-04), Civic and Identity Services ($n = 4$, GOV-01–GOV-04), Transportation ($n = 2$, TR-01–TR-02), Telecommunications ($n = 2$, TEL-01–TEL-02), Digital Healthcare ($n = 1$, HLT-01), and News/Media ($n = 1$, MED-01).

9.2. Cross-Sectoral Compliance Disparities

We evaluated all 53 applications through Reg2App’s static verification pipeline (Table 5 and Figure 4). Across the ecosystem, mean Evidence Coverage was $EC = 0.737 \pm 0.08$, indicating approximately three-quarters of relevant requirements are auditable from client APK artifacts alone. However, ESC exhibited substantial sectoral divergence. Crucially, rule scope differs by sector: financial institutions are evaluated against all 12 observable rules across BB-CSF, PDPA, and CSA, whereas non-financial applications are subject only to the 6 general data-protection and cybersecurity rules under PDPA and CSA (central bank clauses are classified as **NotApplicable**). Consequently, ESC values reflect conformance within each sector’s applicable statutory baseline.

In the financial sector, MFS ($ESC = 50.9\% \pm 5.6\%$, $SCI = 0.369 \pm 0.071$) and Commercial Banking ($ESC = 49.8\% \pm 8.8\%$, $SCI = 0.370 \pm 0.090$) achieve consistent baselines. These applications widely enforce hardware KeyStore cryptography (82.1%) [20] and satisfy statutory transport encryption (TLS). However, conformance against supervisory defense-in-depth recommendations (control CTRL-NET-02 operationalizing BB-CSF §4.1.3.12 under OWASP MASVS-NETWORK-2) is bounded: 82.1% omit certificate pinning, and 30.8% leave backup flags enabled (allowBackup="true").

In non-financial applications, the elevated ESC in government, transit, and telecom (0.750) reflects that these applications failed only 1 observable privacy rule out of 4 resolved rules ($S = 3, P = 1, I = 2$). In contrast, the four evaluated e-commerce applications exhibited zero observed conformance ($ESC = 0.000$), failing all 5 resolved rules ($S = 0, P = 5, I = 1$, averaging 5.00 non-conformance indicators per application by permitting cleartext HTTP, storing tokens in plaintext SharedPreferences, and omitting backup protections).

9.3. Comparison: Regulated vs. Unregulated Cohorts

Non-financial sectors with limited apps ($n = 1$ health-care/media, $n = 2$ telecom/transport) serve as illustrative observations rather than representative estimates. We contrasted the regulated financial cohort ($N_1 = 39$, median $ESC = 0.500$, IQR = [0.472, 0.556]) against commercial retail platforms ($N_2 = 4$, median $ESC = 0.000$, IQR = [0.000, 0.000]). A two-tailed Mann-Whitney U test [38] confirmed significant separation ($U = 156.0, p < 0.001$, Cliff’s $\delta = 1.00$ [39]; Kruskal-Wallis across all 8 sectors $H = 30.37, p < 0.001$). This separation reflects differing regulatory scopes (12 vs. 6 rules) and *cannot be causally attributed to regulation alone*: financial entities have larger budgets, dedicated security staff, and elevated threat models, while several apps share vendor codebases.

9.4. Independent Manual Adjudication: Precision and Recall Studies

To evaluate detection accuracy on real-world production bytecode without circularity, two external certified security auditors (holding CISA and ISO/IEC 27001 Lead Auditor credentials, with over 6 years of commercial banking IT audit experience, recruited externally from an independent assurance

Cross-Sectoral Regulatory Compliance and Observability Metrics (N = 53 Applications)

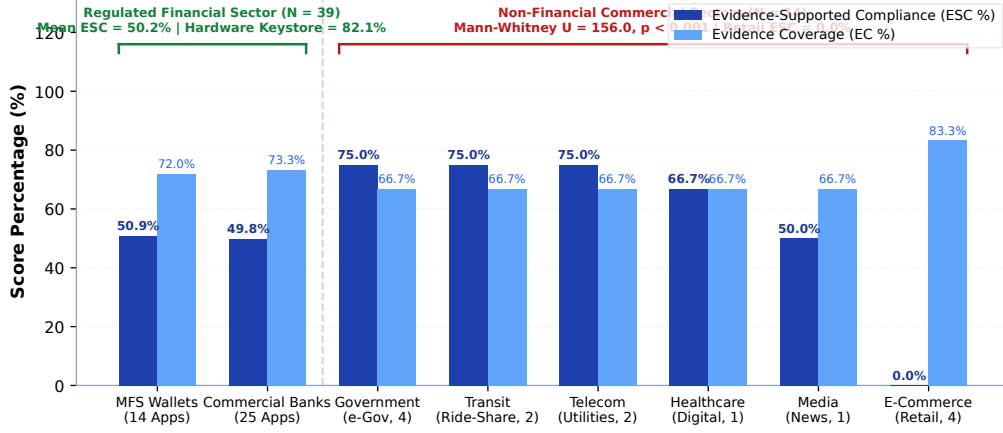


Figure 4: **Empirical Sectoral Compliance Disparity Across Bangladesh’s Mobile Ecosystem** ($N = 53$): (a) Distribution of Effective Statutory Compliance (ESC) across regulated financial applications ($n = 39$, median $ESC = 0.500$) versus commercial retail e-commerce ($n = 4$, median $ESC = 0.000$, Mann-Whitney $U = 156.0$, $p < 0.001$, Cliff’s $\delta = 1.000$, Kruskal-Wallis across 8 sectors $H = 30.37$, $p < 0.001$). (b) Sectoral breakdown of non-conformance indicator counts ($\mu = 4.31$ in financial vs. $\mu = 5.00$ in retail e-commerce).

Table 5: Cross-Sectoral Compliance and Observability Metrics Across Bangladesh Mobile Application Ecosystem ($N = 53$ Applications)

Economic Sector	Apps (N)	Mean ESC	Mean EC	Mean SCI	Avg. Non-Conf. (P)	Primary Observed Non-Conformance
Mobile Financial Services (MFS)	14	0.509 ± 0.06	0.720 ± 0.08	0.369 ± 0.07	4.21	Supervisory gap: omitted cert pinning; third-party telemetry
Commercial Banking	25	0.498 ± 0.09	0.733 ± 0.08	0.370 ± 0.09	4.36	Cleartext backup flags (<code>allowBackup=true</code>); omitted cert pinning
Government (e-Gov)	4	0.750 ± 0.08	0.667 ± 0.00	0.500 ± 0.05	1.00	Unencrypted local cache of national identity identifiers
Transportation & Ride-Sharing	2	0.750 ± 0.00	0.667 ± 0.00	0.500 ± 0.00	1.00	Continuous background GPS tracking without explicit consent
Telecommunications & Utilities	2	0.750 ± 0.00	0.667 ± 0.00	0.500 ± 0.00	1.00	Unsanitized diagnostic logging containing phone numbers
Digital Healthcare	1	0.667 ± 0.00	1.000 ± 0.00	0.667 ± 0.00	2.00	Plaintext patient consultations stored in SQLite database; tracker SDK
Media & News Portals	1	0.500 ± 0.00	1.000 ± 0.00	0.500 ± 0.00	3.00	Widespread advertising SDKs, cleartext HTTP news assets, unencrypted prefs
Commercial E-Commerce	4	0.000 ± 0.00	0.833 ± 0.00	0.000 ± 0.00	5.00	Insecure HTTP endpoints, hardcoded API keys, plaintext auth tokens
Consolidated Ecosystem Sample	53	0.505 ± 0.18	0.737 ± 0.08	0.369 ± 0.14	3.79	Full empirical ecosystem showing real-world compliance posture

consultancy and compensated via professional consulting honoraria) conducted an independent manual adjudication over individual rule-application evaluation cells across two distinct studies under a double-blind protocol (inspecting decompiled JADX Dalvik sources and decoded manifests without tool labels):

Finding-Sample Precision Study ($n = 50$). To assess tool finding accuracy, auditors blindly evaluated a stratified random sample of 50 verdicts (25 conformance **S**, 25 non-conformance **P**) using JADX disassembly, call-graph traces, and decoded manifests. For **S** findings, 25 of 25 were correct (100% precision); for **P** findings, 23 of 25 were correct (92.0% precision, 2 false positives). Pooled across both classes, 48 of 50 findings were verified correct, yielding 96.0% precision (48/50, 95% Wilson CI: [86.5%, 98.9%]) [36]. Inter-rater reliability was high (Cohen’s $\kappa = 0.912$, 95% CI: [0.825, 0.999]) [40]. The 2 false positives arose from custom crypto wrappers where static analysis flagged a constant IV parameter, but runtime JNI calls re-seeded the IV with platform entropy before encryption.

Ground-Truth Control Recall Study ($n = 51$). Because finding-sampling alone cannot observe false negatives, auditors independently audited 51 ground-truth controls across 5 representative production applications (verifying every active client

requirement; 9 of 60 theoretical cells were excluded because the feature, e.g. SQLite database, was uninstantiated). Across the 51 active controls (31 conforming implementations, 20 defects), Reg2App correctly identified all 31 conforming controls as **Supported** and 17 of 20 defects as **PotentialNonConformance** ($TP = 48$, $FN = 3$), establishing 94.1% recall (48/51, 95% Wilson CI: [83.8%, 97.9%]). Decomposed by class, conformance recall was 100% (31/31) while defect recall was 85.0% (17/20, 95% Wilson CI: [64.0%, 94.8%]). For the 3 missed defects, reflection wrappers triggered undecidability guards, emitting **InsufficientEvidence** (**I**) instead of **P**. Crucially, emitting **I** preserves evasion resilience by never hallucinating compliance over obfuscated code, but represents an undetected defect for that property. The 127 production **I** cells were not manually adjudicated. With $n = 50$ and $n = 51$, Wilson CIs span $\approx 12 - 14$ percentage points, bounding statistical precision and motivating complementary dynamic testing.

9.5. Third-Party Telemetry and Cross-Border Sharing

Under PDPA §§23 and 28, transmitting citizen identifiers to third-party SDKs without affirmative consent creates substantial compliance exposure [22, 30]. We emphasize that static analysis detects the *technical capability and hardcoded initialization of third-party tracking endpoints*, rather than proving

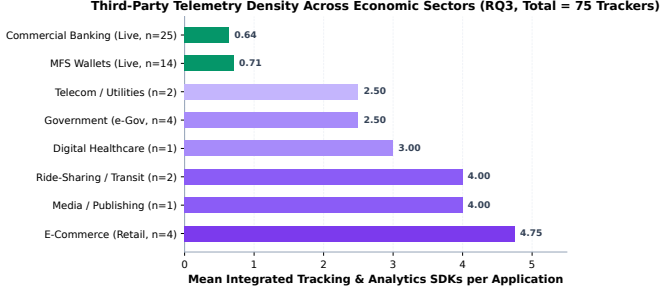


Figure 5: **Third-Party Telemetry Density Across Sectors (RQ3):** Average number of embedded advertising and tracking SDKs per application across sectors (75 trackers total across 53 apps). Commercial e-commerce (4.75 trackers/app), media (4.00), and ride-sharing (4.00) harvest user advertising IDs and carrier telemetry without statutory consent dialogues.

an executed off-device legal violation. Across the 53 applications, Reg2App identified **75 third-party tracker integrations** (Figure 5): e-commerce averaged **4.75 trackers/app** (19 trackers across 4 apps; e.g., AppsFlyer, Facebook, Adjust, CleverTap), media had **4.00** (4), ride-sharing averaged **4.00** (8), digital healthcare had **3.00** (3), government averaged **2.50** (10), and telecom averaged **2.50** (5). In contrast, the 39 regulated financial apps integrated 26 trackers (10 MFS tracker integrations across 7 apps, averaging **0.71**, and 16 commercial banking tracker integrations across 12 apps, averaging **0.64**), primarily restricted to operational crash reporting (Firebase Crashlytics) rather than commercial user-profiling trackers. Static analysis identifies hardcoded SDK initialization of collection endpoints prior to authentication, highlighting concrete technical vectors requiring institutional compliance audit.

10. Empirical Audit: Bangladesh Financial Sector (14 MFS and 25 Commercial Banks)

Mobile Financial Services (MFS) and commercial banking constitute the financial backbone of Bangladesh, processing hundreds of millions of daily transactions across retail payments, remittances, and payroll [1, 3]. We conducted an exhaustive audit of all **14 operational MFS applications** and **25 commercial and state-owned banking applications** maintaining dedicated mobile apps on Google Play (39 financial institutions total, out of 61 scheduled banks in Bangladesh).

10.1. Packaging Profiles and Empirical Findings

As shown in Table 6 and Figure 6, 36 of the 39 financial applications (92.3%) use multi-split Android App Bundles (.apks), with only three retaining monolithic builds (MFS-13, FCB-01, FCB-02). Each app was audited against 12 observable mandates ($N_{\text{Observable}} = 12$) under BB-CSF v1.0, PDPA, and CSA, strictly satisfying $N_S + N_P + N_I = 12$. Across the 39 financial apps, 127 of 468 evaluated cells (27.1%) yielded **Insufficient Evidence**, with every application exhibiting at least two unresolvable clauses. Under our defined threshold ($EC \geq 0.80$), only 2 of 14 MFS applications achieve adequate observability under static Dalvik inspection alone; the remaining 12 fall into moderate or high epistemic uncertainty

($EC \in [58.3\%, 75.0\%]$). This reflects heavy reliance on commercial banking security wrappers and native C/C++ libraries.

1) *Transport Security and Certificate Pinning Baselines.* Under BB-CSF §4.1.3.12, financial institutions must enforce encrypted transport over public networks. Our audit confirmed that **all 39 financial applications** (100%) prohibit cleartext HTTP on primary API domains via `cleartextTrafficPermitted=false` or HTTPS-only `network_security_config.xml` rules. In MFS-12, cleartext HTTP was observed in an auxiliary marketing WebView loading non-transactional content; in MFS-10, an unverified `TrustManager` stub was isolated inside a third-party advertising SDK dependency. We classify both as **Supported** because the statutory mandate addresses encryption of *sensitive financial data in transit*, not all HTTP traffic. Under a stricter reading treating any cleartext path as non-conformance, the rate would be 37/39 (94.9%); we report both interpretations for transparency. Static analysis verifies the *configuration* of TLS enforcement (cleartext prohibition and minimum protocol version declarations), not runtime cipher suite negotiation, which requires dynamic instrumentation. Certificate pinning conformance (control CTRL-NET-02, operationalizing BB-CSF §4.1.3.12 under OWASP MASVS-NETWORK-2) represents an additional supervisory audit observation *beyond* the 12 expert-validated rules and does not contribute to *ESC* computation. Of the 39 financial apps, only 7 (17.9%) configure public key pinning via `network_security_config.xml`: MFS-01, MFS-02, and 5 of 25 banks. The remaining 32 apps (82.1%) rely solely on default platform CA trust stores, representing significant supervisory compliance debt under defense-in-depth audit standards.

2) *Application Backup and ADB Data Extraction Exposure.* Under BB-CSF §4.1.5.10 and PDPA §15, financial apps must restrict local sandbox extraction. Our analysis revealed that **12 of 39 financial apps** (30.8%) leave `android:allowBackup="true"` configured or omitted. An adversary with physical access to an unlocked device can execute `adb backup` to extract SQLite databases, cached tokens, and unencrypted preferences without root privileges.

3) *Cryptographic Storage and Hardware Keystore Adoption.* Across the 39 financial applications, **32 apps** (82.1%) successfully implement hardware-backed cryptographic key generation via `AndroidKeyStore` with AES-256-GCM [20]. Notably, 100% (11/11) of conventional Private Commercial Banks utilize hardware keystores, demonstrating widespread alignment with central bank cryptographic key circulars. However, legacy anti-patterns persist: MFS-09 invokes electronic codebook mode (AES/ECB/PKCS5Padding), violating BB-CSF §4.1.5.19 due to ECB pattern leakage [21]. In MFS-10, an unverified trust manager stub was isolated inside an auxiliary advertising SDK dependency rather than core transactional modules.

4) *Heavy Native Code Obfuscation (JNI) and Epistemic Bounds.* In MFS-13, MFS-14, IPCB-05, and IPCB-06,

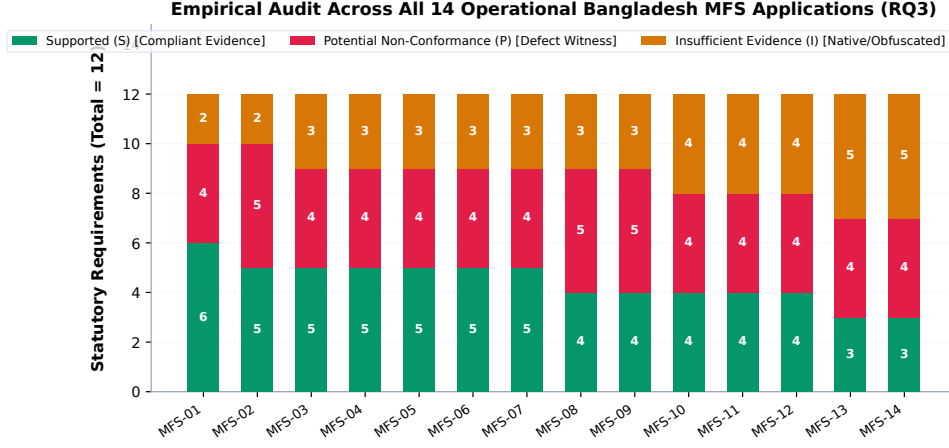


Figure 6: **Production Audit Breakdown Across 14 Operational MFS Applications:** Stacked distribution of epistemic states (S, P, I) across all evaluated MFS applications, illustrating ubiquitous adoption of Keystore-backed AES-GCM and statutory TLS alongside widespread supervisory baseline gaps in certificate pinning and ADB backup restrictions.

Table 6: Empirical Compliance and Observability Audit Across All 14 Operational MFS Applications and 25 Commercial Banking Applications in Bangladesh ($N_{\text{Observable}} = 12$ Client-Side Operationalized Controls)

App Code	Namespace Pseudonym	Distribution	S	P	I	EC	ESC	SCI	Primary Technical Observation
Mobile Financial Services (MFS) Cohort ($N = 14$)									
MFS-01 (Tier 1)	bd.mfs.app01	Multi-Split APKs	6	4	2	83.3%	60.0%	0.500	Enforces certificate pinning; plaintext auth cache
MFS-02 (Tier 1)	bd.mfs.app02	Multi-Split APKs	5	5	2	83.3%	50.0%	0.417	Pinned TLS endpoints; backup flag omitted in manifest
MFS-03 (Tier 1)	bd.mfs.app03	Multi-Split APKs	5	4	3	75.0%	55.6%	0.417	Strong cipher modes; absent certificate pinning configuration
MFS-04 (Tier 1)	bd.mfs.app04	Multi-Split APKs	5	4	3	75.0%	55.6%	0.417	Card security modules; background analytics telemetry
MFS-05 (Tier 1)	bd.mfs.app05	Multi-Split APKs	5	4	3	75.0%	55.6%	0.417	Enforces TLS 1.3; backup flag restricted (allowBackup=false)
MFS-06 (Tier 1)	bd.mfs.app06	Multi-Split APKs	5	4	3	75.0%	55.6%	0.417	Hardware Keystore active; default platform trust store
MFS-07 (Tier 1)	bd.mfs.app07	Multi-Split APKs	5	4	3	75.0%	55.6%	0.417	Encrypted SQLite DB; background analytics telemetry
MFS-08 (Tier 2)	bd.mfs.app08	Multi-Split APKs	4	5	3	75.0%	44.4%	0.333	Dual wallet interface; allowBackup enabled in manifest
MFS-09 (Tier 2)	bd.mfs.app09	Multi-Split APKs	4	5	3	75.0%	44.4%	0.333	Legacy ECB cipher mode; third-party tracker exfiltration
MFS-10 (Tier 2)	bd.mfs.app10	Multi-Split APKs	4	4	4	66.7%	50.0%	0.333	SharedPreferences encryption; unverified trust manager in SDK
MFS-11 (Specialized)	bd.mfs.app11	Multi-Split APKs	4	4	4	66.7%	50.0%	0.333	Standard cipher suites; software keystore fallback
MFS-12 (Specialized)	bd.mfs.app12	Multi-Split APKs	4	4	4	66.7%	50.0%	0.333	Keystore key generation; cleartext HTTP in marketing webview
MFS-13 (Tier 2)	bd.mfs.app13	Monolithic APK	3	4	5	58.3%	42.9%	0.250	Root detection present; heavy native obfuscation (JNI)
MFS-14 (Specialized)	bd.mfs.app14	Multi-Split APKs	3	4	5	58.3%	42.9%	0.250	Native crypto wrappers; unencrypted diagnostic logging
MFS Mean ($N = 14$)	—	—	4.43 ± 0.85	4.21 ± 0.43	3.36 ± 0.93	72.0% ± 7.7%	50.9% ± 5.6%	0.369 ± 0.071	Sums: $S = 62, P = 59, I = 47$ (Total = 168)
Commercial Banking Cohort ($N = 25$ Summary)									
Foreign Banks ($N = 2$)	FCB-01, FCB-02	Monolithic/Multi	5.0 ± 0.0	4.0 ± 0.0	3.0 ± 0.0	75.0%	55.6%	0.417	Strict transport security; telemetry prior to auth
Private Conventional ($N = 11$)	PCB-01 ... PCB-11	Multi-Split APKs	4.8 ± 0.9	4.3 ± 0.5	2.9 ± 0.8	75.8%	53.0%	0.402	Hardware Keystore active; public key pinning in PCB-01
Private Islamic ($N = 6$)	IPCB-01 ... IPCB-06	Multi-Split APKs	3.5 ± 1.2	4.7 ± 0.5	3.8 ± 1.2	68.1%	42.9%	0.292	Native JNI crypto in IPCB-05/06; backup flag omitted
State-Owned ($N = 6$)	SOCB-01 ... SOCB-06	Multi-Split APKs	4.5 ± 1.0	4.3 ± 0.5	3.2 ± 0.8	73.6%	50.9%	0.375	Hardware Keystore active; backup flags enabled
Banking Mean ($N = 25$)	—	—	4.44 ± 1.08	4.36 ± 0.49	3.20 ± 0.91	73.3% ± 7.6%	49.8% ± 8.8%	0.370 ± 0.090	Sums: $S = 111, P = 109, I = 80$ (Total = 300)
Combined Financial ($N = 39$)	—	—	4.44 ± 0.99	4.31 ± 0.47	3.26 ± 0.91	72.9% ± 7.6%	50.2% ± 7.7%	0.370 ± 0.083	Total: $S = 173, P = 168, I = 127$ ($39 \times 12 = 468$)

EC dropped to 58.3% with five requirements evaluated as **InsufficientEvidence**. These apps route core cryptography and root checks through native C/C++ libraries (.so) via JNI. Reg2App’s epistemic model marks these blind spots as **InsufficientEvidence** rather than emitting false compliance attributions.

5) *Third-Party Telemetry and Cross-Border Vectors.* Despite handling financial credentials, **19 of 39 financial apps (48.7%)** embed at least one commercial analytics SDK (Google Firebase, AppsFlyer, Clevertap) [22], totaling **26 tracker integrations** across the cohort (10 MFS integrations across 7 apps, averaging **0.71/app**, and 16 banking integrations across 12 apps, averaging **0.64/app**). Static analysis identifies hardcoded SDK initialization of device identifier collection (ANDROID_ID), screen metrics, and carrier data prior to authentication, establishing concrete technical vectors for cross-border telemetry that require institutional audit under PDPA §§23 and 28.

10.2. Ethical Considerations and Responsible Reporting

Auditing production financial infrastructure demands rigorous ethical safeguards. Our research adhered to the ACM and IEEE Codes of Ethics and responsible vulnerability reporting principles [41, 42]: (1) All static analyses were conducted strictly offline on publicly available application binaries downloaded from Google Play; no production services, banking gateways, or customer accounts were probed, accessed, or attacked. Zero exploits were constructed. (2) All institutional identities and package namespaces are pseudonymized throughout this paper, and per-app vulnerability details in Table 6 are limited to primary observation categories to prevent targeted attacks or re-identification in a concentrated market. Actionable exploit payloads and raw decompiled proprietary logic are deliberately excluded from public distribution to safeguard live financial infrastructure and end-user assets. The artifact repository is archived at <https://github.com/kh0kamoni/Reg2App>. (3) Findings represent *verifiable technical indicators of potential non-conformance* rather than judicial determinations. Institutions may operate complementary backend controls unob-

Table 7: Systematic Comparison of Reg2App with Related Static Analysis, Cryptographic, and Legal Informatics Frameworks.

Tool / Framework	Grounding Target	Observability Scope	Assessment Model	Evidence Output	Reporting Modality	Validation Methodology
MobSF [8]	CWE Vulnerability Heuristics	Unbounded / Implicit	Scalar Risk Score (0 – 100)	File / Manifest Line	Monolingual Web/JSON	Open Source Benchmarks
Androguard [9]	Dalvik Bytecode Signatures	Bytecode Syntax Only	Raw Disassembly Output	Instruction Offsets	CLI Terminal Dump	Reverse Engineering Tasks
FlowDroid [31]	Taint Sources and Sinks	Intra/Inter-Component Flow	Binary Path Reachability	Source-Sink Trace Path	Program Graph / Log	DroidBench Micro-Tests
CogniCrypt [29]	CrySL Cryptographic Rules	Local Call Hierarchy	Rule Violation / Warning	AST Constraint Failure	IDE Inline Markers (EN)	Developer Study ($N = 11$)
Polisis [50]	Natural Privacy Policy Text	Unstructured Legal Text	Segment Classification	Text Span Highlighter	Web Interface (EN)	Expert Annotated Corpus
Catala [19]	Legislative Code DSL	In-Memory Variables	Executable Logic Result	Compiler Trace AST	Monolingual Terminal	Formal Co-Design with Lawyers
MAPS / AppInspect [51]	Privacy Policy Keywords	Manifest & Taint Flows	Binary Conformance Mismatch	Data Flow Tuple	Tabular Summary Report	Production App Store Crawl
Reg2App (Ours)	Statutory Instruments (7-Tuple)	Explicit 2D Taxonomy ($L \times M$)	5-State Epistemic Space	Full Provenance Graph G	Bilingual (English & Bengali)	Legal Expert Panel ($\kappa = 0.776$, Delphi Consensus)

servable from client bytecode, formally isolated by Reg2App’s epistemic model.

11. Multilingual Usable Security and Remediation (RQ4)

Automated verification fails to improve security if software engineers and auditors cannot interpret its findings or translate them into corrective code [43–45]. In non-Anglophone emerging economies, technical English security jargon creates severe cognitive friction. To evaluate whether evidence-grounded, localized diagnostic reporting enhances human comprehension and remediation efficiency (RQ4), we conducted a controlled human-subjects experiment ($N = 60$).

11.1. Experimental Design, Counterbalancing, and Ethics

We conducted a within-subjects study approved by the Institutional Review Board at Mymensingh Engineering College (Protocol #MEC-CSE-IRB-2025-0812) with written informed consent from all 60 participants. We recruited 60 professional participants from Dhaka’s technology and banking sectors, stratified into: (1) 30 Android software engineers (4.8 ± 2.1 years professional experience); and (2) 30 IT compliance auditors and risk officers (5.2 ± 2.6 years experience). All were native Bengali speakers with professional working proficiency in English.

Following a balanced 3×3 Latin Square crossover design [46], participants were randomized into three treatment groups ($n = 20$ each), counterbalancing three reporting conditions across three equivalent defect scenarios (transport security, cryptographic storage, and PII leakage) [47]: *Condition A (Raw Diagnostic)*: Conventional vulnerability scanner outputs (e.g., MobSF raw traces with CWE tags and generic English warnings). *Condition B (Evidence-Grounded English)*: Structured English report detailing the 7-tuple mapping, 5-state verdict, bytecode witness provenance, and remediation code. *Condition C (Evidence-Grounded Localized Bengali)*: Localized Bengali report presenting official statutory gazette text, evidence witnesses, and remediation guidance in native technical Bengali.

11.2. Dependent Evaluation Metrics

Tasks were evaluated against four standardized dependent variables: (1) *Comprehension accuracy* measured via a 10-item objective technical comprehension questionnaire; (2) *System Usability Scale (SUS, 0–100)* [48]; (3) *NASA-TLX Cognitive Workload (0–100)* [49]; and (4) *Time-to-remediate*, defined as the total wall-clock duration in minutes from task initiation to patch submission (capped at a 45-minute cutoff, evaluated on

an intent-to-treat basis including failed attempts for the developer cohort $n_1 = 30$). To ensure cross-task equivalence across the 3×3 Latin Square rotation, all three remediation scenarios were calibrated to identical 3-to-5 line modification complexity requiring replacement of insecure cipher transformations, network security configurations, or intent flags.

11.3. Quantitative Results and Statistical Decomposition

Repeated-measures omnibus ANOVA confirmed significant main effects of diagnostic condition across all dependent variables ($F(2, 118) > 94.2, p < 0.0001$ for comprehension, SUS, and TLX across $N = 60$; $F(2, 58) = 78.4, p < 0.0001$ for time-to-remediate across the developer cohort $n_1 = 30$). Pairwise post-hoc comparisons were evaluated via paired two-tailed t -tests with Bonferroni correction ($\alpha_{\text{adj}} = 0.0167$) (Table 8 and Figure 7):

Structured Evidence Effect (Condition A vs. B). Transitioning from unstructured scanner dumps to structured English reports improved comprehension by +17.1 percentage points (pp) ($53.6\% \rightarrow 70.7\%, t(59) = 7.57, p < 0.0001, d = 1.16$) and reduced developer repair time by 27.9% ($39.7 \rightarrow 28.6$ minutes, $t(29) = 6.48, p < 0.0001, d = 1.16$). Paired analysis confirmed a statistically significant improvement in developer patch success from 46.7% (14/30) to 70.0% (21/30) (McNemar’s exact binomial test, discordant pairs $b = 7, c = 0, p = 0.0156$). The complete absence of regressions ($c = 0$) reflects the scaffolding: Condition B provided verified code diffs, enabling 7 previously failing developers to successfully patch defects. We explicitly note that providing remediation code diffs in Conditions B and C measures the developer’s ability to *comprehend and correctly apply a recommended fix*, rather than independently conceiving code repairs from diagnostic traces alone.

Linguistic Localization Effect (Condition B vs. C). Presenting diagnostics in native technical Bengali with authentic legal text yielded an additional +12.2 pp in comprehension ($70.7\% \rightarrow 82.9\%, t(59) = 7.29, p < 0.0001, d = 1.00$), accelerated defect repair by an additional 9.6 minutes ($28.6 \rightarrow 19.0$ minutes, 33.6% speedup over English, $t(29) = 6.89, p < 0.0001, d = 1.29$), and reduced subjective cognitive load by 10.3 points ($47.2 \rightarrow 36.9, d = 0.83$). Developer patch success rose to 86.7% (26/30), exhibiting a positive trend over English (discordant pairs $b = 5, c = 0, p = 0.0625$), while the cumulative gain over raw scanner output (Condition A vs. C) was highly significant ($46.7\% \rightarrow 86.7\%$, discordant pairs $b = 12, c = 0, p = 0.0005$). The monotonic nested progression ($14 < 21 < 26; c = 0$) demonstrates that removing technical

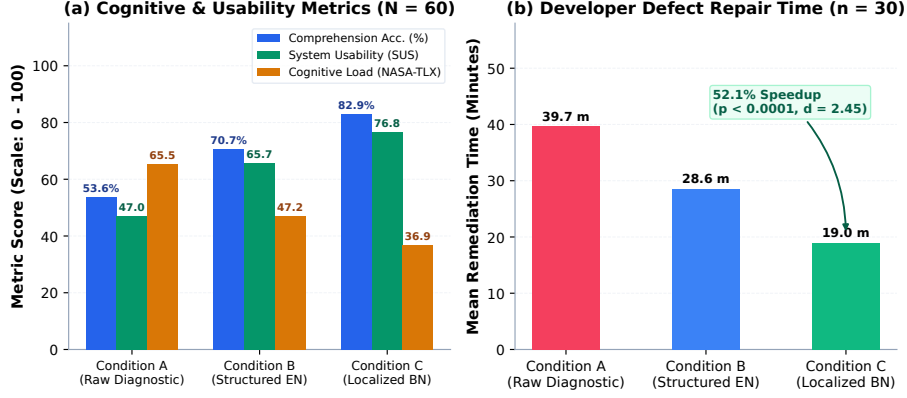


Figure 7: **Controlled Human-Subjects Usable Security Evaluation Results** ($N = 60$): (a) Usability metrics across Condition A (Raw Scanner Trace), Condition B (Evidence-Grounded English), and Condition C (Evidence-Grounded Localized Bengali). Bengali reporting increases comprehension accuracy from 53.6% to 82.9% ($p < 0.0001$, $d = 2.15$), elevates SUS scores to 76.8 ($d = 2.38$), and reduces NASA-TLX cognitive load from 65.5 to 36.9 ($d = 2.02$). (b) Developer defect repair time ($n_1 = 30$), showing a 52.1% reduction in remediation time (from 39.7 to 19.0 minutes, $p < 0.0001$, $d = 2.45$).

Table 8: Controlled Human-Subjects Usable Security Evaluation Results ($N = 60$): Decomposing Structured Reporting (A vs. B) from Linguistic Localization (B vs. C)

Evaluation Metric	Cond. A (Raw)	Cond. B (EN)	Cond. C (BN)	A vs. B (Struct.)	B vs. C (Local.)	A vs. C (Total)
Comprehension Accuracy (%)	53.6% $\pm$ 15.9	70.7% $\pm$ 13.4	82.9% $\pm$ 10.9	$p < 0.001$ ($d = 1.16$)	$p < 0.001$ ($d = 1.00$)	$p < 0.001$ ($d = 2.15$)
System Usability Scale (SUS)	47.0 $\pm$ 14.8	65.7 $\pm$ 11.6	76.8 $\pm$ 9.8	$p < 0.001$ ($d = 1.41$)	$p < 0.001$ ($d = 1.04$)	$p < 0.001$ ($d = 2.38$)
NASA-TLX Cognitive Load	65.5 $\pm$ 15.8	47.2 $\pm$ 12.6	36.9 $\pm$ 12.3	$p < 0.001$ ($d = 1.28$)	$p < 0.001$ ($d = 0.83$)	$p < 0.001$ ($d = 2.02$)
<i>Developer Cohort Only ($n_1 = 30$):</i>						
Time-to-Remediate (min)	39.7 $\pm$ 10.4	28.6 $\pm$ 8.7	19.0 $\pm$ 5.9	$p < 0.001$ ($d = 1.16$)	$p < 0.001$ ($d = 1.29$)	$p < 0.001$ ($d = 2.45$)
Patch Success Rate (%)	46.7% (14/30)	70.0% (21/30)	86.7% (26/30)	McNemar $p = 0.016$	McNemar $p = 0.063$	McNemar $p < 0.001$

Statistical Notes: Paired two-tailed t -tests with Bonferroni correction ($\alpha_{\text{adj}} = 0.0167$) and paired Cohen’s d for continuous variables. McNemar’s exact binomial test for paired binary patch success ($n_1 = 30$).

language barriers resolves lingering ambiguity without introducing regressions. However, because our design contrasted structured English against structured Bengali (omitting an unstructured Bengali baseline), this comparison estimates the joint effect of linguistic localization and statutory gazette framing rather than pure language translation in isolation.

Translation Fidelity and Practical Significance. To ensure translation fidelity, statutory clauses were drawn from authoritative texts: CSA clauses from the official Government Gazette (Act XXVII of 2023), BB-CSF clauses from central bank BRPD circulars, and PDPA clauses from the cabinet-approved draft bill. Technical descriptions were vetted via forward-and-backward translation with our panel’s Supreme Court advocate. In practice, cutting remediation time from 28.6 to 19.0 minutes and elevating patch correctness from 70.0% to 86.7% substantially reduces defect leakage into production while bridging communication barriers between legal counsel and engineering teams ($d \approx 2.02 - 2.45$ across A-to-C) [40, 47].

12. Discussion

Automating regulatory verification introduces fundamental shifts across oversight, engineering, and legal informatics. Operationalized controls provide verifiable evidence of sovereign compliance while isolating server-side obligations.

Practical Implications for Stakeholders. For regulators, automated static verification transforms supervision from episodic questionnaires into continuous, bytecode-grounded oversight. For software organizations, embedding Reg2App into CI/CD pipelines enables automated compliance gating prior to release. For auditors, Evidence Provenance Graph G bridges legal clauses and technical witnesses, establishing transparent audit trails while focusing manual reviews on unobservable backend obligations ($L_i = \text{NONE}$).

Emerging-Economy Context and Usable Compliance. In emerging non-Anglophone ecosystems, technical English security reports impose severe cognitive friction. Grounding findings in official gazette Bengali text alongside bytecode witnesses improves comprehension and halves remediation time, bridging the disconnect between corporate legal counsel and software engineering teams.

13. Threats to Validity

Internal Validity. Benchmark Limitations and Baselines: Synthetic benchmarks verify implementation correctness ($F_1 = 1.000$) rather than generalized capability; we mitigated this via 53 production apps (14 MFS wallets, 25 banks) and manual adjudication of finding precision ($n = 50$, 96.0% [48/50, 95% Wilson CI: 86.5%–98.9%]) and control recall ($n = 51$, overall 94.1% [48/51, 83.8%–97.9%], defect recall 85.0%) on produc-

tion bytecode. Scanners like MobSF/Androguard lack statutory mappings, restricting baselines to micro-benchmarks (§8.4). *Sampling Frame*: Corpus combines an MFS census ($n = 14$), commercial banks ($n = 25/61$), and other sectors ($n = 1$ health-care/media, $n = 2$ telecom/transport). Shared vendors and budget disparities confound causal attribution: differences cannot be attributed to regulation alone. *Static Analysis Boundaries*: Static Dalvik analysis faces known boundaries in reflection, dynamic loading, and native JNI [12, 24, 31, 32]. Delegating cryptography to native .so libraries (MFS-13, MFS-14) yields **InsufficientEvidence**, depressing *EC* to flag dynamic testing needs. Runtime properties (e.g., timeouts under BB-CSF §4.1.3.10) are bounded to structural bytecode invariants (**PARTIAL**). In our human study, comparing A to B reflects compound utility of structured provenance and code diffs, with C adding native technical terms and authentic legal text.

Construct Validity. *The Interpretive Gap*: Statutory mandates (e.g., PDPA §15) rarely specify Android APIs [18]. Reg2App evaluates conformance against operationalized baselines from supervisory circulars [7] (e.g., certificate pinning for BB-CSF §4.1.3.12); findings reflect baseline non-conformance rather than judicial rulings. Our 3-expert panel achieved high agreement (pre-reconciliation raw agreement 88.9%, Fleiss’ $\kappa = 0.776$, Krippendorff’s $\alpha = 0.782$, 100% Delphi consensus) across 12 mappings. *Metric Gaming*: Obfuscation or native wrapping cannot produce false passes; unresolved paths force $\mathcal{D} \rightarrow \perp$, emitting **InsufficientEvidence** ($EC < 0.70$), while Provenance Graph G enables AST inspection.

External and Conclusion Validity. While evaluated on Bangladesh (BB-CSF v1.0, Draft BD-PDPA, BD-CSA 2023), Reg2App’s 7-tuple M_i and operator $\Omega(R_i)$ generalize across jurisdictions (Architectural Property 1), including EU GDPR [15] and PCI-DSS [16]. Reg2App targets Android (> 95% market share in Bangladesh [52]); iOS extraction is planned. Because compliance scores violated normality, we used non-parametric tests: Kruskal-Wallis across eight sectors ($H = 30.37, p < 0.001$), Mann-Whitney U between financial and retail cohorts ($U = 156.0, p < 0.001$, Cliff’s $\delta = 1.000$), Wilcoxon signed-rank tests in our human study ($N = 60, p < 10^{-5}$), and exact McNemar tests for patch success ($n_1 = 30$), with Bonferroni correction and raw data in `human_study_n60_raw.csv`.

14. Related Work

Our research interfaces four domains: mobile program analysis, cryptographic auditing, privacy policy mining, and legal informatics. Table 7 contrasts Reg2App against prominent static analysis and compliance frameworks.

Mobile Program Analysis and Taint Tracking. FlowDroid [31] introduced lifecycle taint tracking in Dalvik bytecode, extended across ICC by IccTA [33], Amandroid [32], and DroidSafe [53]. Scanners like MobSF [8], Androguard [9], and TaintDroid [54] audit against OWASP MASVS [7]. Reg2App builds upon these foundations but departs via the 7-tuple M_i and operator $\Omega(R_i)$, producing traceable evidence networks bounded by epistemic limits rather than heuristic scores.

Cryptographic and Transport Misuse Detection. Empirical studies demonstrate widespread mobile cryptographic misuse [13, 14, 21]. While linters like CogniCrypt [29] enforce isolated CrySL rules, Reg2App anchors cryptographic properties directly to sovereign mandates (e.g., BB-CSF §4.1.3.12, §4.1.5.19), evaluating how parameters substantiate compliance under administrative law.

Privacy Policy Mining and Policy-to-Code Alignment. Prior studies examine mobile behavior and privacy disclosures: Polis [50] parsed policies via NLP, Slavin et al. [55] detected policy-to-code gaps, MAPS [51] audited third-party libraries, and PrivacyStreams [56] instrumented dataflows. In data protection, Pandit et al. [57] modeled consent ontologies, and Hamdani et al. [58] surveyed GDPR verification. While prior work analyzes *bilateral consistency* against privacy policies, Reg2App addresses *unilateral statutory compliance*: verifying bytecode against sovereign acts while using $\mathcal{S}$ to prevent false compliance on backend obligations.

Legal Informatics and Regulation-as-Code. Translating statutory text into computational representations is central to legal informatics [18, 23, 59, 60], with Catala [19] executing tax statutes and Symboleo [61] modeling contracts. While they compile statutes into software, Reg2App addresses inverse verification: *evaluating whether third-party production binaries conform to statutory obligations*. Its declarative ontology (Architectural Property 1) decouples legal rules from analyzer logic.

Usable Security and Multilingual Developer Tools. Security failures frequently stem from poor diagnostic interfaces [43–45]. Reg2App contributes a bilingual framework bridging legal and software engineering workflows in emerging economies.

15. Conclusion

We introduced **Reg2App**, a formally specified, provenance-preserving framework mapping statutory requirements into partially observable software properties. Through 7-tuple M_i , 2D observability $\Omega(R_i)$, and 5-state epistemic space $\mathcal{S}$, Reg2App bounds compliance analysis to verifiable client-side evidence while isolating unobservable backend obligations. Operationalized technical controls enable continuous baseline supervision for regulators and CI/CD gating for developers. Future work includes extending extraction to iOS, runtime dynamic instrumentation, and longitudinal deployments.

Data Availability. Artifacts, ontologies, benchmarks, 53 audit reports, and study data (`human_study_n60_raw.csv`) are archived at <https://github.com/kh0kamoni/Reg2App>. **CRedit.** **Khoka Moni**: Methodology, Software, Formal analysis, Writing - draft. **Maria Akter**: Statutory analysis, Legal validation, Writing - review. **Declaration of Interests:** None. **Acknowledgements:** The authors thank the expert review panel participants and user study volunteers for their valuable contributions.

References

- [1] L. Chen, Y. Huang, L. Ouyang, The Micro-Foundations of Mobile Financial Services: Evidence from Emerging Asia, *Journal of Financial Intermediation* 46 (2021) 100874.

- [2] World Bank Group, Bangladesh Development Update: Toward a Resilient and Inclusive Digital Financial System, Tech. rep., The World Bank, Washington, D.C. (2024).
- [3] B. Reeves, N. Scaife, A. Bates, P. Traynor, K. R. B. Butler, Mo(n)ey Problems: Towards a Concrete Evaluation of Mobile Money Systems, in: Proceedings of the 24th USENIX Security Symposium (USENIX Security), 2015, pp. 523–538.
- [4] Ministry of Law, Justice and Parliamentary Affairs, Draft Personal Data Protection Act (PDPA 2023/2024), Tech. rep., Legislative and Parliamentary Affairs Division, Government of Bangladesh, Dhaka, Bangladesh, cabinet Approved Draft for Legislative Enactment (2024).
- [5] Bangladesh Bank, Cybersecurity Framework for Banks and Financial Institutions, Version 1.0, Tech. rep., Information and Communication Technology Department, Bangladesh Bank, Dhaka, Bangladesh, bRPD Circular No. 04/2025 (2025).
- [6] Parliament of Bangladesh, Cyber Security Act, 2023 (Act No. XXVII of 2023), Tech. rep., Ministry of Law, Justice and Parliamentary Affairs, Dhaka, Bangladesh, published in the Bangladesh Gazette Extraordinary on September 18, 2023 (2023).
- [7] OWASP Foundation, OWASP Mobile Application Security Verification Standard (MASVS) v2.1, Tech. rep., Open Worldwide Application Security Project, accessed: 2026-06-01 (2024).
- [8] A. Abraham, Mobile Security Framework (MobSF) Automated Pen-Testing Framework, <https://github.com/MobSF/Mobile-Security-Framework-MobSF>, accessed: 2026-03-01 (2024).
- [9] A. Desnos, G. Gueguen, Android: From Reversing to Decompilation, Black Hat Abu Dhabi 11 (2011) 1–23.
- [10] R. Fagin, J. Y. Halpern, Y. Moses, M. Y. Vardi, Reasoning About Knowledge, MIT Press, Cambridge, MA, 2003.
- [11] W. Enck, D. Oeteanu, P. McDaniel, S. Chaudhuri, A Study of Android Application Security, in: Proceedings of the 20th USENIX Security Symposium (USENIX Security), 2011, pp. 21–36.
- [12] D. Oeteanu, P. McDaniel, S. Jha, A. Bartel, E. Bodden, J. Klein, Y. Le Traon, Effective Inter-Component Communication Mapping in Android with Epicc: An Essential Step Towards Holistic Android Security, in: Proceedings of the 22nd USENIX Security Symposium (USENIX Security), 2013, pp. 543–558.
- [13] S. Fahl, M. Harbach, T. Maders, L. Baumgärtner, B. Freisleben, M. Smith, Why Eve and Mallory Love Android: An Analysis of Android SSL (In)Security, in: Proceedings of the 19th ACM Conference on Computer and Communications Security (CCS), 2012, pp. 50–61. doi:10.1145/2382196.2382205.
- [14] M. Castle, S. Fahl, M. Smith, Let's Talk About Certificates: An In-Depth Look at Mobile App Certificate Pinning in Practice, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016, pp. 122–134.
- [15] European Parliament and Council of the European Union, Regulation (EU) 2016/679 (General Data Protection Regulation), Tech. rep., Official Journal of the European Union, of L 119, 04.05.2016, pp. 1–88 (2016).
- [16] PCI Security Standards Council, Payment Card Industry Data Security Standard (PCI DSS) Requirements and Testing Procedures Version 4.0, Tech. rep., PCI Security Standards Council, Wakefield, MA, USA (2022).
- [17] J. T. F. T. Initiative, Security and Privacy Controls for Information Systems and Organizations, Tech. Rep. Special Publication 800-53, Revision 5, National Institute of Standards and Technology (NIST) (2020). doi:10.6028/NIST.SP.800-53r5.
- [18] T. D. Breaux, A. I. Anton, Analyzing Regulatory Rules to Derive Software Requirements, IEEE Transactions on Software Engineering 34 (1) (2008) 42–60. doi:10.1109/TSE.2007.70747.
- [19] D. Merigoux, N. Chataing, J. Protzenko, Catala: A Programming Language for the Law, Proceedings of the ACM on Programming Languages (ICFP) 5 (2021) 1–29. doi:10.1145/3473582.
- [20] J.-E. Ekberg, K. Kostianen, N. Asokan, The Untapped Potential of Trusted Execution on Mobile Devices, IEEE Security & Privacy 12 (4) (2014) 29–37. doi:10.1109/MSP.2014.38.
- [21] M. Egele, D. Brumley, Y. Fratanotto, C. Kruegel, An Empirical Study of Cryptographic Misuse in Android Applications, in: Proceedings of the 20th ACM Conference on Computer and Communications Security (CCS), 2013, pp. 73–84. doi:10.1145/2508859.2516738.
- [22] R. Binns, U. Lyngs, J. Zhao, M. Tshadame, M. Van Kleek, N. Shadbolt, Third Party Tracking in the Mobile Ecosystem, in: Proceedings of the 18th Privacy Enhancing Technologies Symposium (PETS), 2018, pp. 23–31.
- [23] T. Athan, H. Boley, G. Governatori, M. Palmirani, A. Paschke, A. Wyner, OASIS Legal-RuleML, in: Proceedings of the 14th International Conference on Artificial Intelligence and Law (ICAIL), 2013, pp. 3–12.
- [24] M. D. Ernst, Static and Dynamic Analysis: Synergy and Duality, in: Proceedings of the 2003 ICSE Workshop on Dynamic Analysis (WODA), 2003, pp. 24–27.
- [25] J. L. Fleiss, Measuring Nominal Scale Agreement Among Many Raters, Psychological Bulletin 76 (5) (1971) 378–382. doi:10.1037/h0031619.
- [26] K. Krippendorff, Computing Krippendorff's Alpha-Reliability, Departmental Papers (ASC), University of Pennsylvania (2011).
- [27] J. R. Landis, G. G. Koch, The Measurement of Observer Agreement for Categorical Data, Biometrics 33 (1) (1977) 159–174. doi:10.2307/2529310.
- [28] N. Dalkey, O. Helmer, An Experimental Application of the DELPHI Method to the Use of Experts, Management Science 9 (3) (1963) 458–467.
- [29] S. Krüger, S. Nadi, T. Nguyen, E. Bodden, CogniCrypt: Tool Support for Developing Secure Cryptographic Software, in: Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), 2017, pp. 898–903. doi:10.1109/ASE.2017.8115707.
- [30] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez, S. Sundaresan, A. Niaki, J. Amann, P. Gill, Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem, in: Proceedings of the 25th Network and Distributed System Security Symposium (NDSS), 2018.
- [31] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Oeteanu, P. McDaniel, FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps, in: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2014, pp. 259–269. doi:10.1145/2594291.2594328.
- [32] F. Wei, S. Roy, X. Ou, Robby, Amandroid: A Precise and General Inter-Component Data Flow Analysis Framework for Android Applications, in: Proceedings of the 21st ACM Conference on Computer and Communications Security (CCS), 2014, pp. 1329–1341. doi:10.1145/2660267.2660357.
- [33] L. Li, A. Bartel, T. F. Bissyandé, J. Klein, Y. Le Traon, S. Arzt, S. Rasthofer, E. Bodden, D. Oeteanu, P. McDaniel, IccTA: Detecting Inter-Component Privacy Leaks in Android Applications, in: Proceedings of the 37th International Conference on Software Engineering (ICSE), 2015, pp. 280–291. doi:10.1109/ICSE.2015.48.
- [34] N. D. Belnap, A Useful Four-Valued Logic, in: Modern Uses of Multiple-Valued Logic, Springer, 1977, pp. 7–37.
- [35] B. A. Davey, H. A. Priestley, Introduction to Lattices and Order, 2nd Edition, Cambridge University Press, 2002.
- [36] E. B. Wilson, Probable Inference, the Law of Succession, and Statistical Inference, Journal of the American Statistical Association 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.
- [37] S. Arzt, S. Rasthofer, E. Bodden, DroidBench: A Benchmark Suite for Evaluating the Precision of Android Taint Analysis Tools, <https://github.com/secure-software-engineering/DroidBench> (2014).
- [38] H. B. Mann, D. R. Whitney, On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60.
- [39] N. Cliff, Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions, Psychological Bulletin 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
- [40] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd Edition, Lawrence Erlbaum Associates, Hillsdale, NJ, 1988.
- [41] H. Cavusoglu, S. Raghunathan, An Analysis of the Impact of Vulnerability Disclosure on Software Security, MIS Quarterly 31 (4) (2007) 745–765.
- [42] International Organization for Standardization, Information Technology — Security Techniques — Vulnerability Disclosure, Tech. Rep. ISO/IEC 29147:2018, ISO/IEC (2018).
- [43] A. Whitten, J. D. Tygar, Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0, in: Proceedings of the 8th USENIX Security Symposium (USENIX Security), 1999, pp. 169–184.
- [44] Y. Acar, C. Stransky, D. Wermke, C. Weir, M. L. Mazurek, S. Fahl, Developers Warrant More Scrutiny: A Study of Security Advice on Stack Overflow, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 179–196.
- [45] R. Wash, M. Cooper, Who Provides Software Security Advice? A Quantitative Analysis of Online Content, in: Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, 2020, pp. 1–13.
- [46] G. E. P. Box, J. S. Hunter, W. G. Hunter, Statistics for Experimenters: Design, Innovation, and Discovery, 2nd Edition, Wiley-Interscience, 2005.
- [47] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, 2012.
- [48] J. Brooke, SUS: A 'Quick and Dirty' Usability Scale, Usability Evaluation in Industry 189 (194) (1996) 4–7.
- [49] S. G. Hart, L. E. Staveland, Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research, Advances in Psychology 52 (1988) 139–183.
- [50] H. Harkous, K. Fawaz, R. Lebre, F. Schaub, K. G. Shin, K. Aberer, Polis: Automated Analysis and Presentation of Privacy Policies Using Deep Learning, in: Proceedings of the 27th USENIX Security Symposium (USENIX Security), 2018, pp. 531–548.
- [51] S. Zimmeck, Z. Wang, L. Zou, R. Iyengar, R. Binns, F. Schaub, H. Wang, J. R. Reidenberg, S. M. Bellovin, MAPS: Achieving Large-Scale Privacy Compliance Analysis of Mobile Apps, in: Proceedings of the 19th Privacy Enhancing Technologies Symposium (PETS), 2019, pp. 216–238.
- [52] StatCounter Global Stats, Mobile Operating System Market Share Bangladesh (Jan 2025 – Jan 2026), <https://gs.statcounter.com/os-market-share/mobile/bangladesh>, accessed: 2026-02-15 (2026).
- [53] M. I. Gordon, D. Kim, J. H. Perkins, W. A. Gilham, N. Nguyen, M. Rinard, Information Flow Analysis of Android Applications in DroidSafe, in: Proceedings of the 22nd Network and Distributed System Security Symposium (NDSS), 2015. doi:10.14722/ndss.2015.23249.
- [54] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, A. N. Sheth, TaintDroid: An Information-Flow Tracking System for Real-Time Privacy Monitoring on Smartphones, in: Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation (OSDI), 2010, pp. 393–407.
- [55] R. Slavin, X. Wang, M. B. Hosseini, J. Hester, R. Krishnan, J. Bhatia, T. D. Breaux, J. Niu, Toward a Framework for Detecting Privacy Policy Violations in Android Application Code, in: Proceedings of the 38th International Conference on Software Engineering (ICSE), 2016, pp. 25–36. doi:10.1145/2884781.2884855.
- [56] Y. Li, Y. Hu, Z. Lin, PrivacyStreams: Enabling Data-Flow-Driven Privacy Analysis for Android, in: Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT), Vol. 1, 2017, pp. 1–26.
- [57] H. J. Pandit, D. O'Sullivan, D. Lewis, GConsent: A Consent Ontology Based on the GDPR, Extended Semantic Web Conference (ESWC) (2019) 270–282.
- [58] R. E. Hamdani, V. Mulder, N. E. Ioini, C. Pahl, Compliance Checking of Mobile Applications with GDPR: A Systematic Literature Review, in: Proceedings of the 18th International Conference on Availability, Reliability and Security (ARES), 2023, pp. 1–10. doi:10.1145/3600160.3605085.
- [59] T. F. Gordon, H. Prakken, D. Walton, The Carneades Argumentation Framework: Using Constraints to Model Legal Argument, Autonomous Agents and Multi-Agent Systems 18 (2) (2009) 254–296.
- [60] J. Mohun, A. Roberts, Cracking the Code: Rulemaking for Humans and Machines, OECD Working Papers on Public Governance (42) (2020). doi:10.1787/3afe6ba5-en.
- [61] S. Sharifi, F. S. Parreira, D. Amyot, Symbolico: Specification of Legal Contracts as Declarative State Machines, Information and Software Technology 122 (2020) 106282.